2026/09/16

대한상공회의소

RV32i_CPU_AP

BFinal Project

발표 : 1 조

Contents

Overview 01

System Architecture 02

Memory Map 03

RV32i-Multi Cycle 04

UVM Verification 05

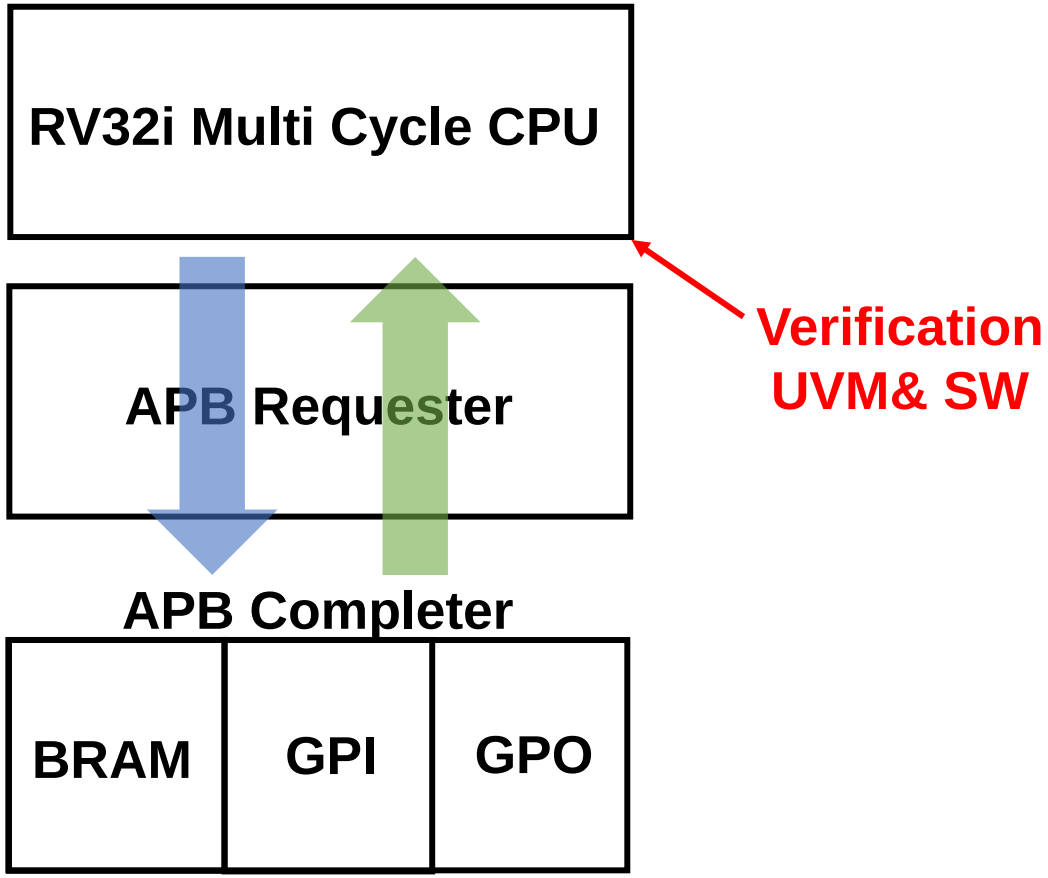
APB Requester & Completer 06

SW Verification 07

Troubleshooting, Conclusion and Discussion 08

01. Overview

■ 개요



■ 역할

김기원

- RV32i Multi Cycle 설계 및 검증

김민식

- CPU UVM 검증

김태수

- APB Requester & Completer 설계 및 검증

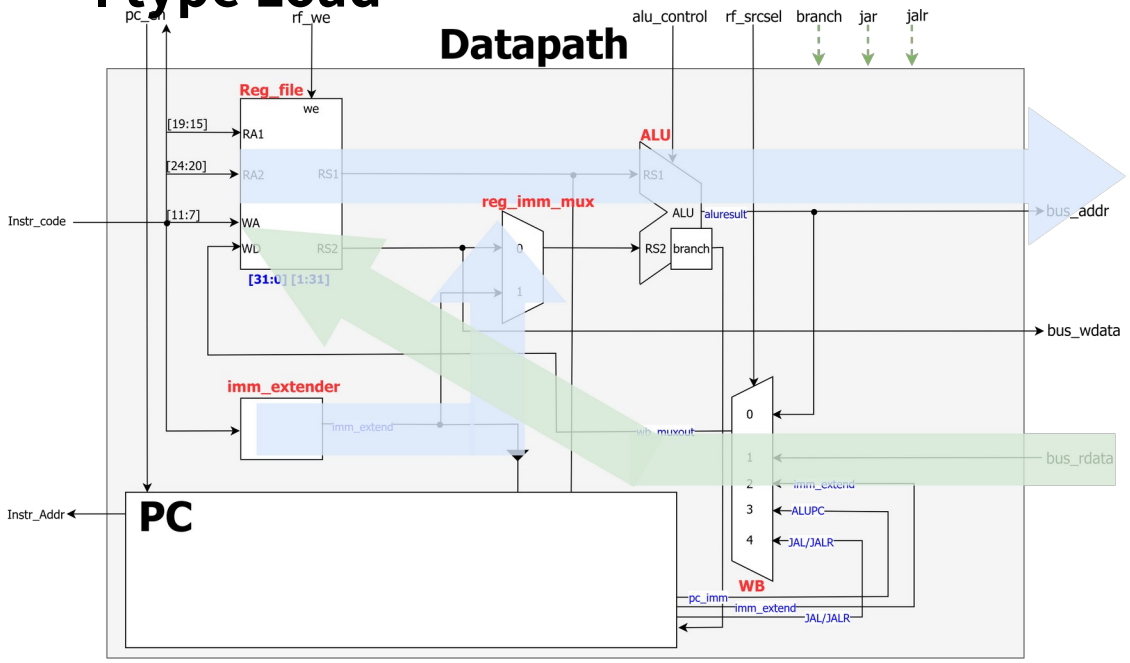
정명훈

- CPU SW 검증

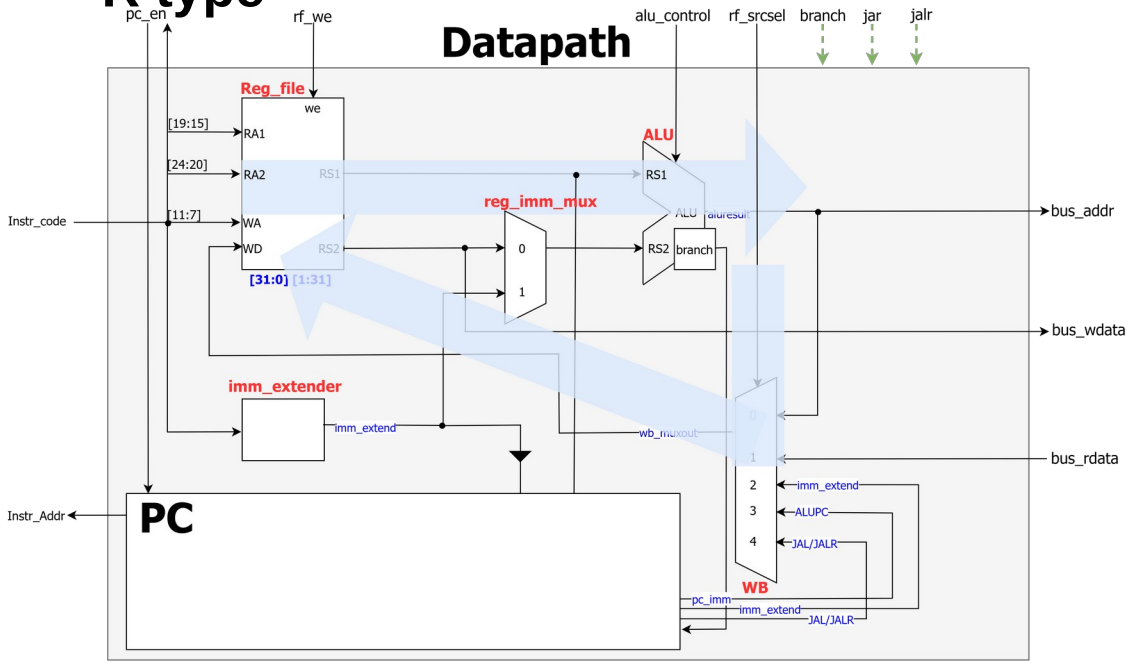
01. Overview

■ 목적 Why ? Single -> Multi Cycle

I type Load



R type





가장 긴 Propagation Delay 기준으로 고정된 클럭 주기
→ 짧은 명령어일수록 Time slack 손해

01. Overview

■ 목적 How ? Single -> Multi Cycle

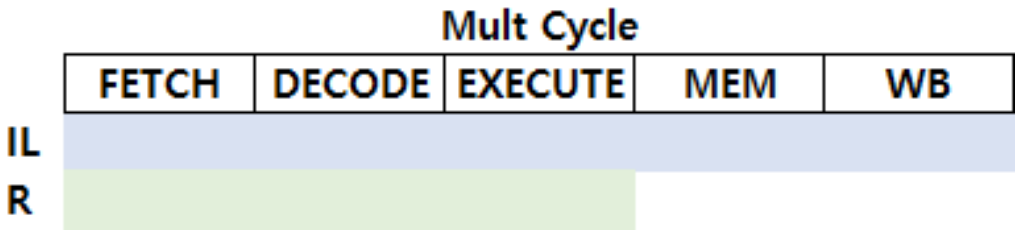
Multi-cycle State 구성 & 사이클 비교

STATE	OPERATION
FETCH	PC 값 갱신
DECODE	레지스터 파일 읽기
EXECUTE	ALU 연산
MEM	데이터 메모리 접근
WB	레지스터 파일 쓰기

장점

(1) 각 stage 의 Propagation Delay 가 짧아짐
→ Time slack 확보

(2) 확보된 Time slack 만큼 클럭 주기 ↓
= 주파수를 높일 수 있음

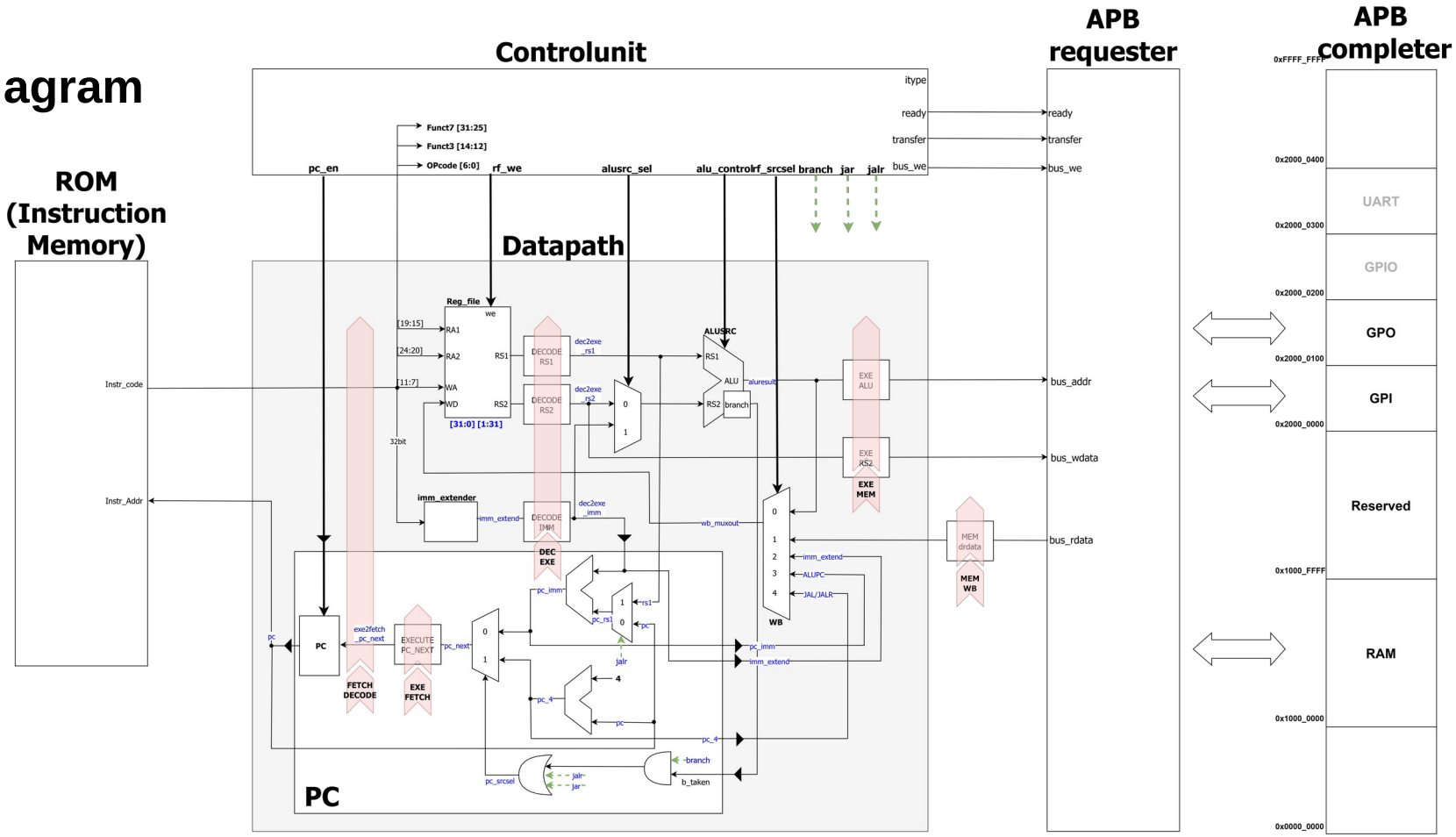


VS



02. System Architecture

Block Diagram



명령어 저장하는 메모리

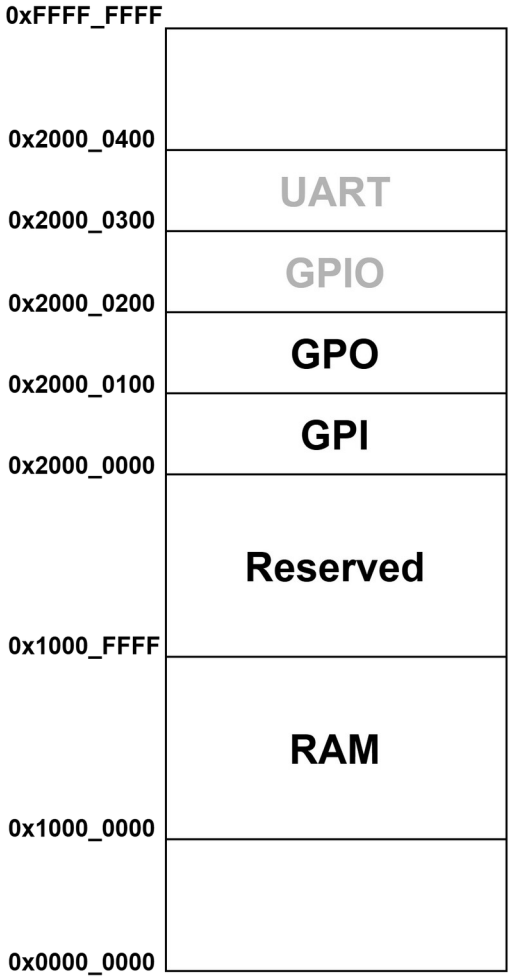
명령어 해석 & 실행 연산장치

버스 프로토콜 변환

메모리 & 주변장치

03. Memory Map

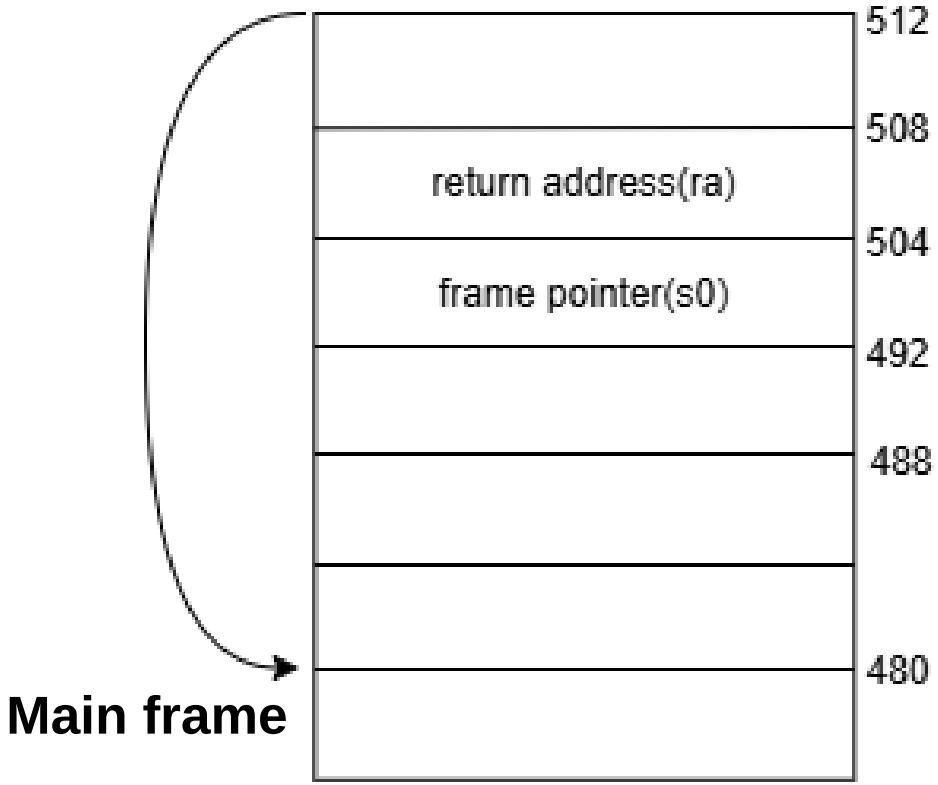
Memory Address Map



주소 범위	영역	용량
0x1000_0000 ~ 0x1000_FFFF	RAM	64 KB
0x1001_0000 ~ 0x1FFF_FFFF	Reserved	256 MB
0x2000_0000 ~ 0x2000_00FF	GPI	256B
0x2000_0100 ~ 0x2000_01FF	GPO	256B
0x2000_0200 ~ 0x2000_02FF	GPIO	256B
0x2000_0300 ~ 0x2000_03FF	UART	256B

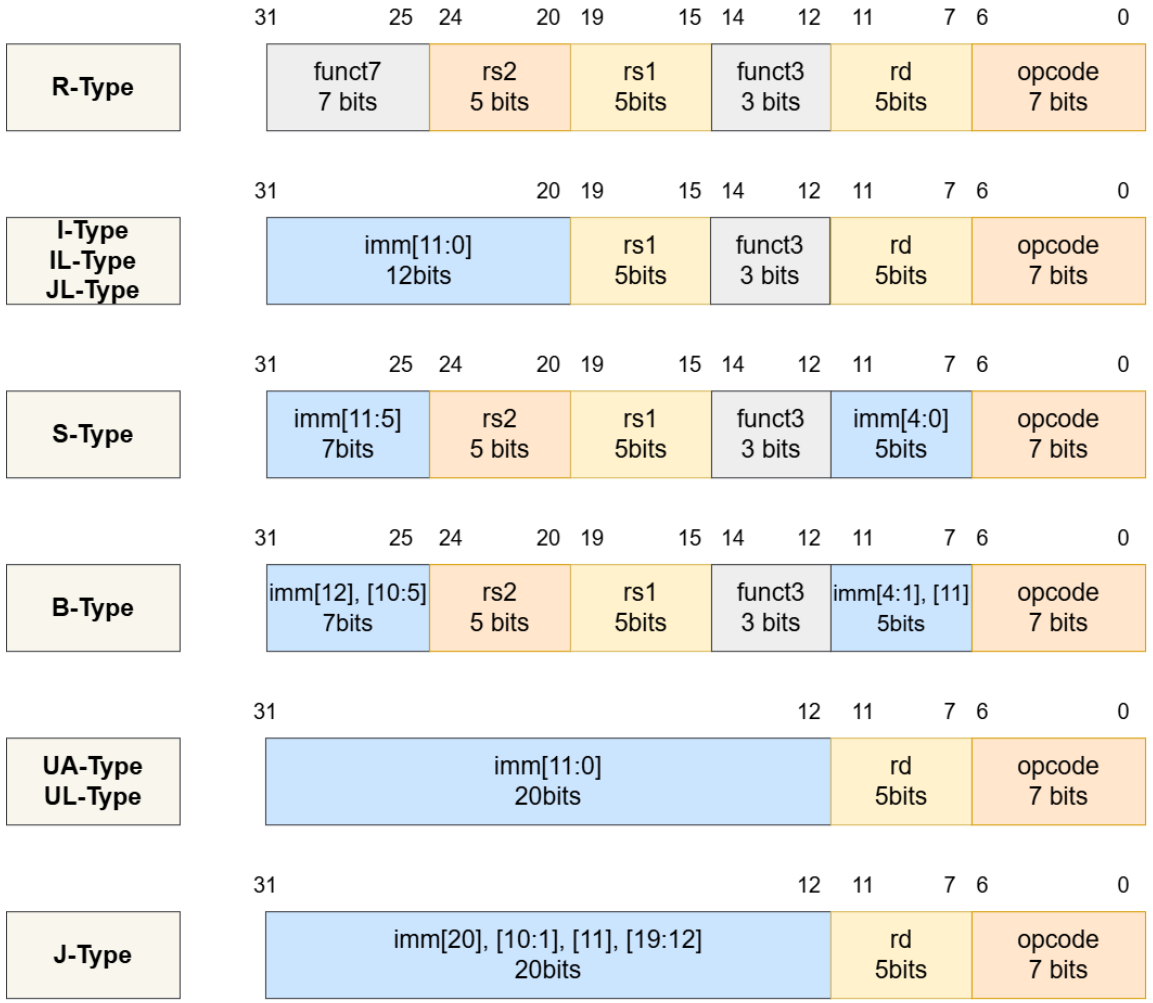
03. Memory Map

■ Stack Point



PC 주소	Hex 코드	RISC-V 어셈블리	동작 및 설명
0x00	20000113	addi sp, zero, 512	스택 포인터 초기화 (sp = 512)
0x04	fe010113	addi sp, sp, -32	main 스택 프레임 32 바이트 할당 (sp = 480)
0x08	00112e23	sw ra, 28(sp)	스택 주소 508 에 ra 저장 (480 + 28)
0x0C	00812c23	sw s0, 24(sp)	스택 주소 504 에 s0 저장 (480 + 24)
0x10	02010413	addi s0, sp, 32	프레임 포인터 설정 (s0 = 512)

04. RISCV RV32i-Multi Cycle



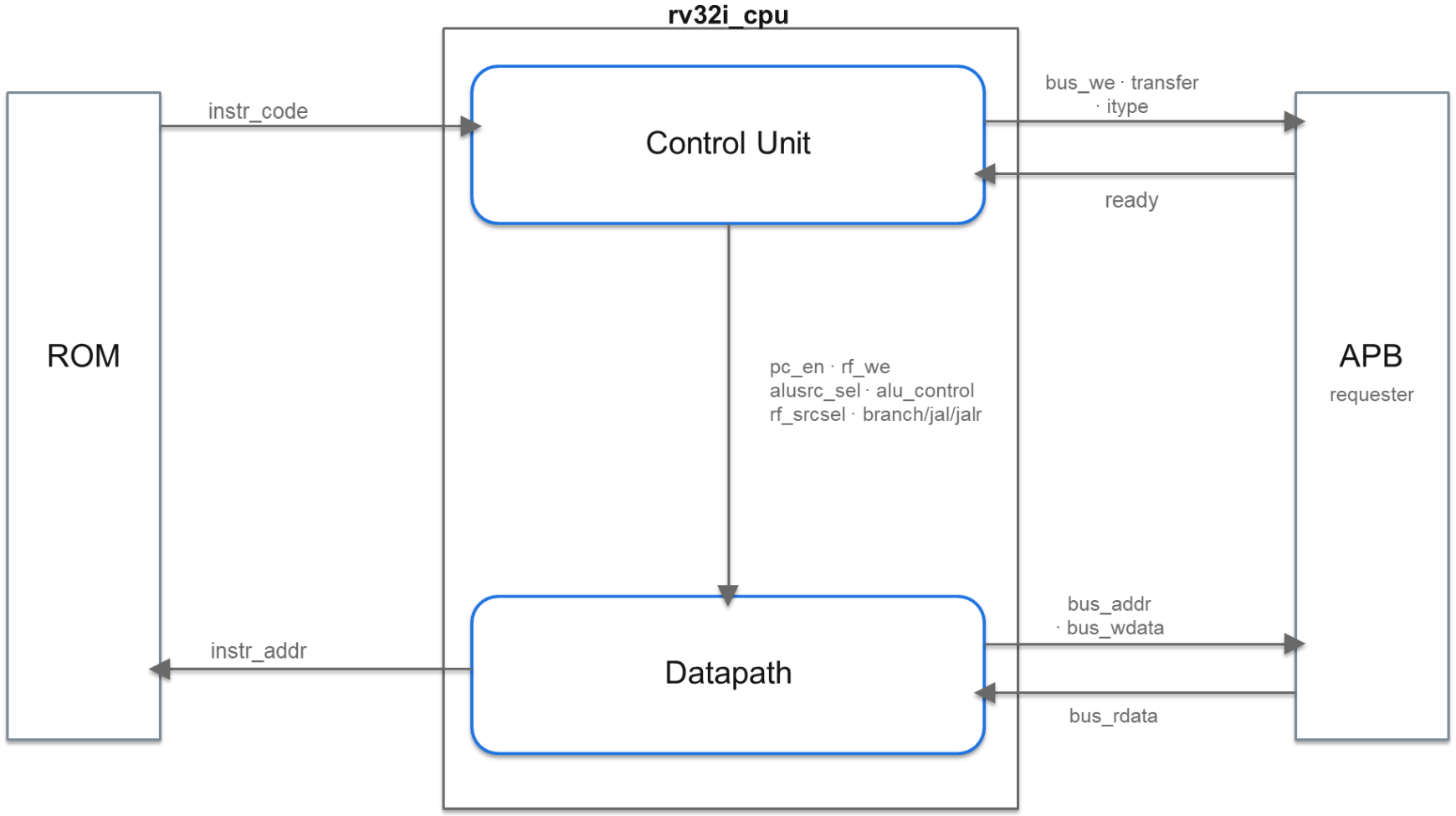
RISCV

- 오픈 소스 기반 32 비트 고정 길이 하드웨어 명령어 세트
- 용도에 따라 6 가지 타입으로 구분

명령어 Type 별 특징

Type	특징
R-type	rs1, rs2 두 레지스터 값을 연산해서 rd 에 저장 (레지스터 - 레지스터 연산) => 산술 , 비교 , 시프트 연산
I-type	rs1 과 12 비트 immediate 를 연산 , rd 에 저장 (레지스터 - 즉치값 연산 , load 포함)
S-type	store 명령어용 . immediate 가 두 조각 (imm[11:5], imm[4:0]) 으로 나뉘어 있는 게 특징
B-type	branch 명령어 . if 문 , for 문 , while 문 같은 조건 분기문 컴파일시 사용
U-type	20 비트 immediate 로 상위 비트를 채우는 명령어
J-type	jump 명령어

04. RISCV RV32i-Multi Cycle



하버드 아키텍처 : ROM 과 Memory(주변장치) 접근 신호를 분리한 아키텍처 . ROM 과 주변 장치 동시 접근 가능 .

신호 흐름 : ROM -> instr_code -> Control Unit 명령어 해석 -> Datapath 제어 , 결과는 APB 버스로 외부 전달

04. RISCV RV32i-Multi Cycle

Multi Cycle FSM



Multi Cycle : 명령어 하나를 **여러 클럭 사이클에 나눠서 처리**하는 방식

상태별 역할

- **FETCH**: pc_en=1, 명령어 인출
- **DECODE**: 레지스터 파일 읽기 , 명령어 해석
- **EXECUTE**: ALU 연산 / 주소 연산 분기 판정
- **MEM**: APB 버스를 통한 메모리 접근 (읽기 , 쓰기)
- **WB**: 레지스터 파일 쓰기

04. RISCV RV32i-Multi Cycle

제어 신호

신호명	설명
pc_en	PC 레지스터 갱신 인에이블 (FETCH 에서만 1)
rf_we	레지스터 파일 쓰기 인에이블
alu_srcsel	ALU 두 번째 입력 선택 (0=rs2, 1=immediate)
rf_srcsel	WB(Write-Back) 단계에서 WB_MUX 를 통해 레지스터 파일에 쓸 값 선택을 위한 신호
branch, jal, jarl	branch, jal, jarl 명령어 선택 신호
bus_we	APB 를 통한 쓰기 여부
bus_addr	APB 주소 신호
bus_wdata	APB 를 통해 전달할 데이터
ready	APB Requester 완료 신호 (MEM/WB 의 대기 해제 조건 , fetch 상태 전환)

04. RISCV RV32i-Multi Cycle

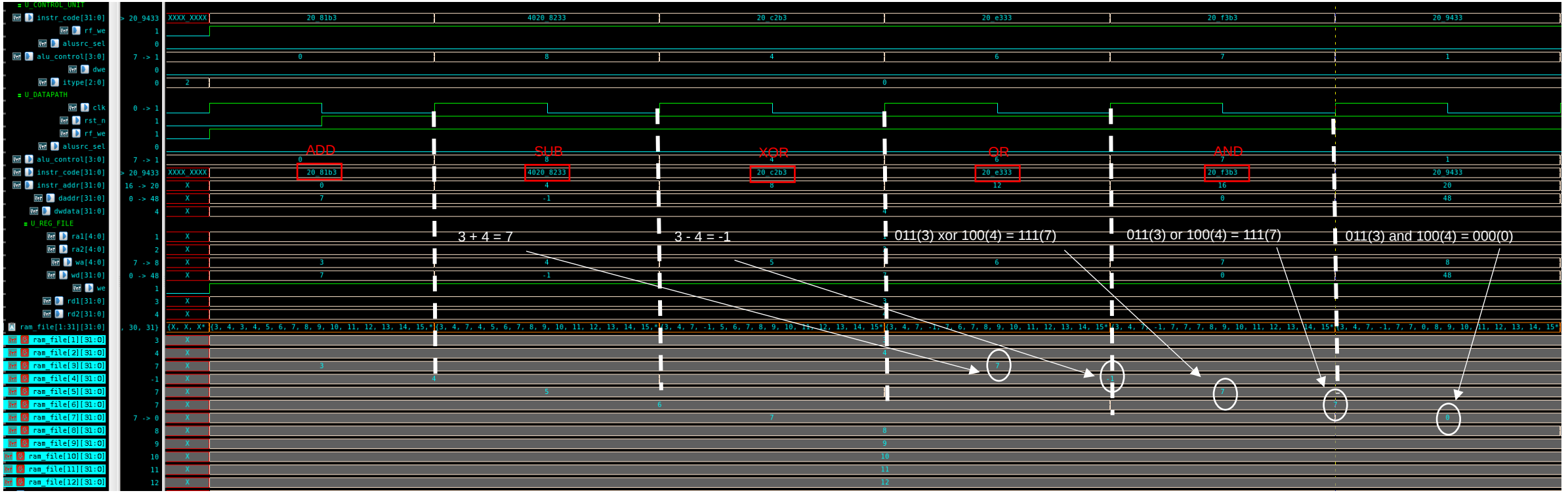
R-type 시뮬레이션 시나리오

Inst	fucn7	rs2	rs1	func3	rd	opcode	instr_rom
ADD	0000_000	0_0010	0_0001	000	0_0011	011_0011	h'0020_81b3
SUB	0100_000	0_0010	0_0001	000	0_0100	011_0011	h'4020_8233
XOR	0000_000	0_0010	0_0001	100	0_0101	011_0011	h'0020_c2b3
OR	0000_000	0_0010	0_0001	110	0_0110	011_0011	h'0020_e333
AND	0000_000	0_0010	0_0001	111	0_0111	011_0011	h'0020_f3b3
SLL	0000_000	0_0010	0_0001	001	0_1000	011_0011	h'0020_9433
SRL	0000_000	0_0010	0_0001	101	0_1001	011_0011	h'0020_d4b3
SRA	0100_000	0_0010	0_0001	101	0_1010	011_0011	h'4020_d533
SLT	0000_000	0_0010	0_0001	010	0_1011	011_0011	h'0020_a5b3
SLTU	0000_000	0_0010	0_0001	011	0_1100	011_0011	h'0020_b633

04. RISCV RV32i-Multi Cycle

R-type 시뮬레이션

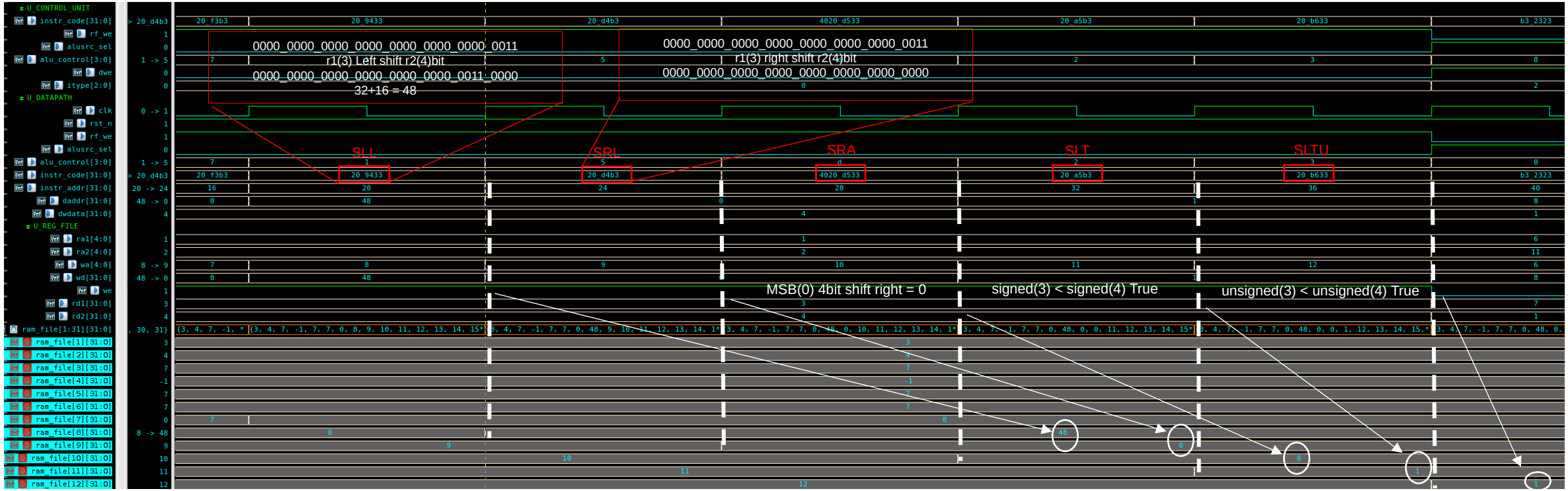
Register 의 [1] = 3, [2] = 4 로 초기화



04. RISCV RV32i-Multi Cycle

R-type 시뮬레이션

Register 의 [1] = 3, [2] = 4 로 초기화



04. RISCV RV32i-Multi Cycle

R-type 시뮬레이션 시나리오

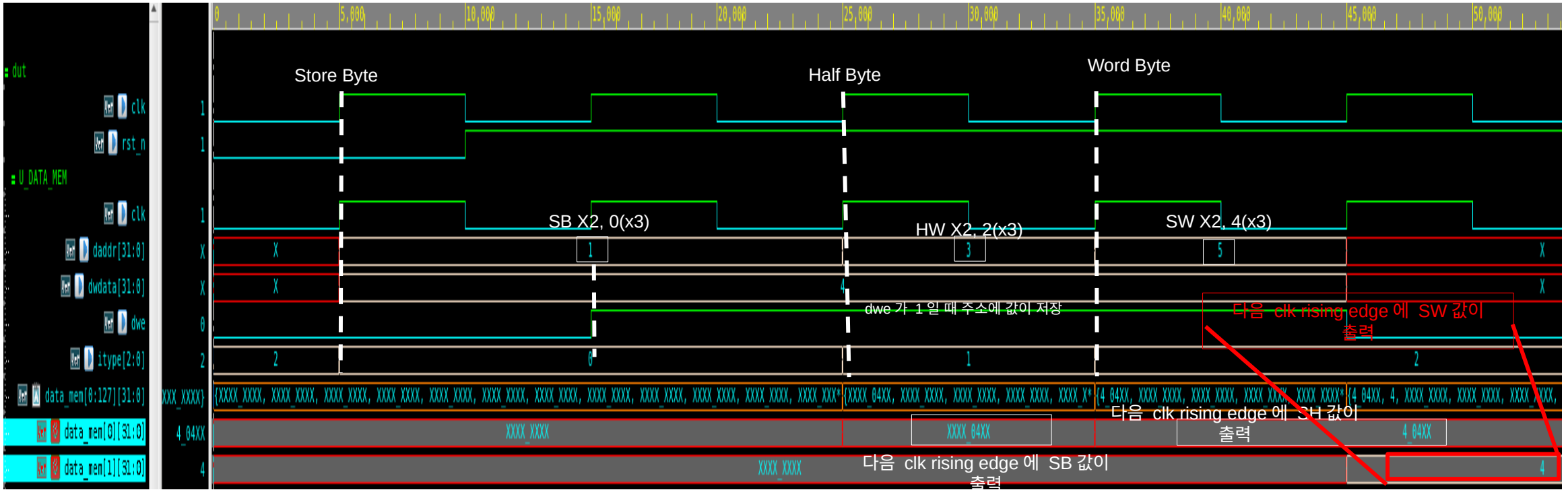
Register Address(X3) 에 연산 값 (-3 + 4) 저장

Inst	fucn7	rs2	rs1	func3	imm	opcode	instr_rom
SB	0000_000	0_0010	0_0011	000	0_0000	010_0011	h'0020_81b3
SH	0000_000	0_0010	0_0011	001	0_0010	010_0011	h'4020_8233
SW	0000_000	0_0010	0_0011	100	0_0100	010_0011	h'0020_c2b3

04. RISCV RV32i-Multi Cycle

R-type 시뮬레이션

Register 의 [1] = -3, [2] = 4 로 초기화

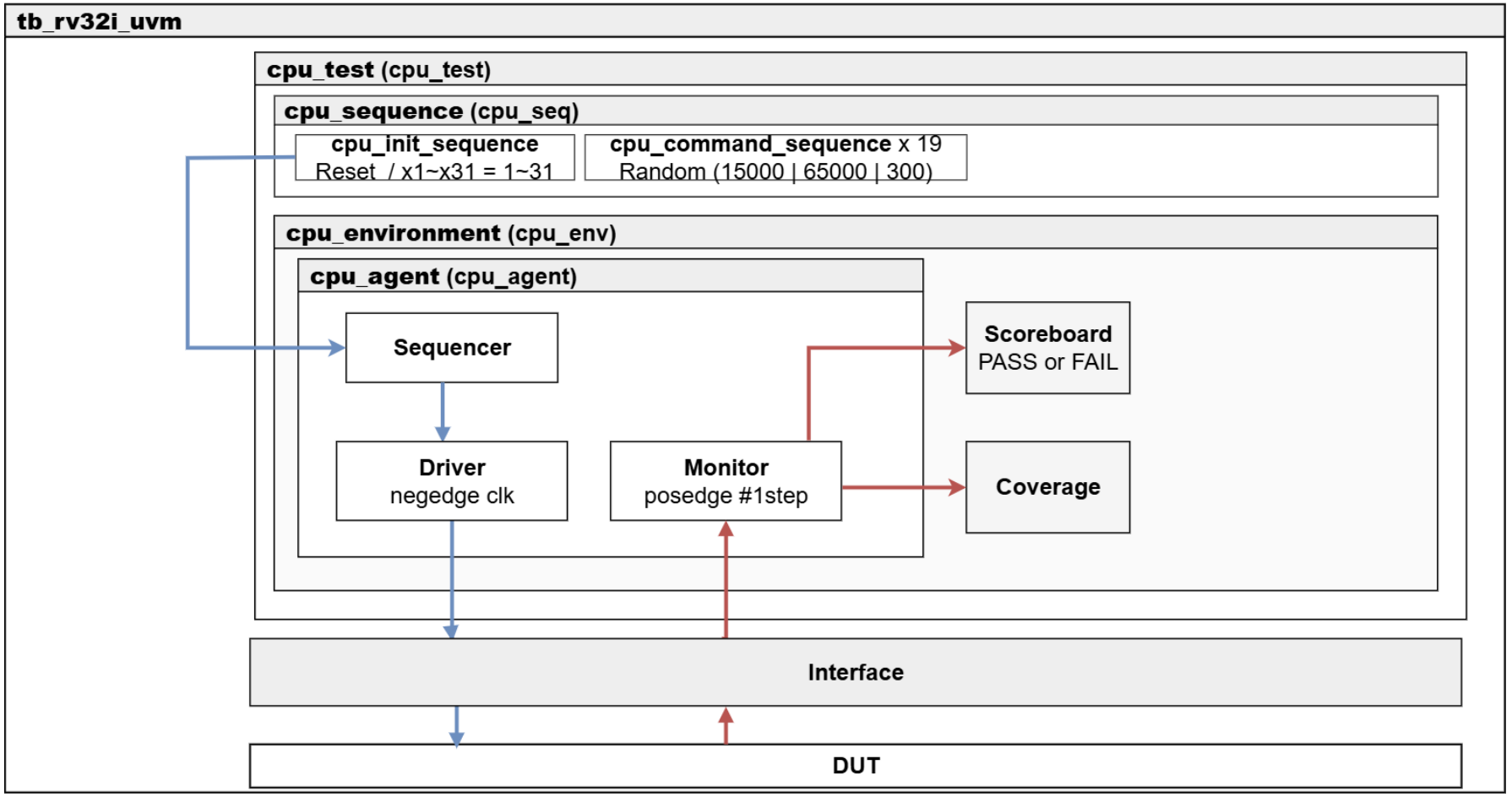


05. UVM Verification

■ R/I Type 명령어

총 19 개 , 각 명령어 시작 전 init_sequence 에서 reset & 레지스터 초기값 설정

RV32I UVM Testbench Architecture



05. UVM Verification

R/I Type

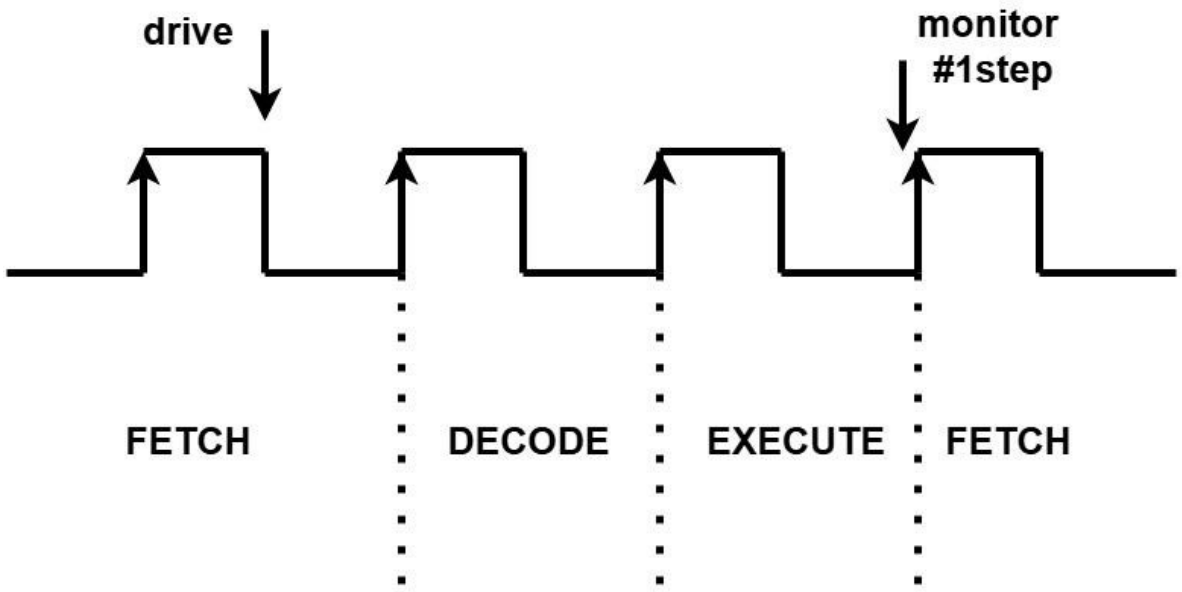
opcode	Type	command	연산
0110011(0x33)	R	ADD	rd = rs1 + rs2
0110011(0x33)	R	SUB	rd = rs1 - rs2
0110011(0x33)	R	SLL	rd = rs1 << rs2 [4:0]
0110011(0x33)	R	SLT	rd = ((signed) rs1 < (signed) rs2) ? 1:0
0110011(0x33)	R	SLTU	rd = (rs1 < rs2)?1:0
0110011(0x33)	R	XOR	rd = rs1 ^ rs2
0110011(0x33)	R	SRL	rd = rs1 >> rs2 [4:0]
0110011(0x33)	R	SRA	rd = (signed)rs1 >>> rs2 [4:0]
0110011(0x33)	R	OR	rd = rs1 | rs2
0110011(0x33)	R	AND	rd = rs1 & rs2
0000011(0x03)	I	ADDI	rd = rs1 + imm

05. UVM Verification

■ Timing

Dirve : negedge clk
Monitor : #1step posedge clk

daddr (alu_result) : posedge clk #1



■ Pass 조건

4 개 신호의 제어 신호 조건이 모두 만족시 pass 판단

monitor 가 매 클럭의 상승에지 마다 값을 체크함

pass 와 fail 은 3 사이클 단위로 판단함

제어 신호	조건	설명
rf_we	1	레지스터 write
alusrc_sel	0 : R (rs2), 1 : I (imm)	타입별 소스 선택
alu_control	명령어에 맞는 control 신호	명령어별 alu 연산 선택
daddr(alu_result)	expexted alu_result 일치	실제 연산 결과와 예상 결과 일치

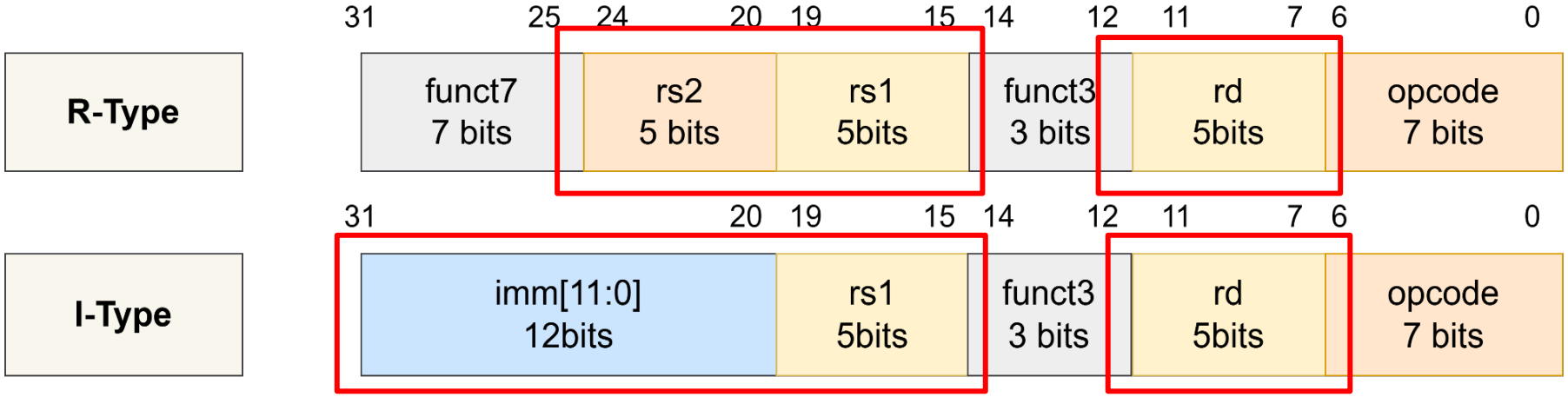
05. UVM Verification

■ Coverage

- reset : 0,1
- R-type : 10 개
- I-type : 일반 6 개 , shift 3 개
- rs1, rs2 : 0~31
- imm : signed 12 bit [-2048~2047]
- rd : 0~31

■ Cross

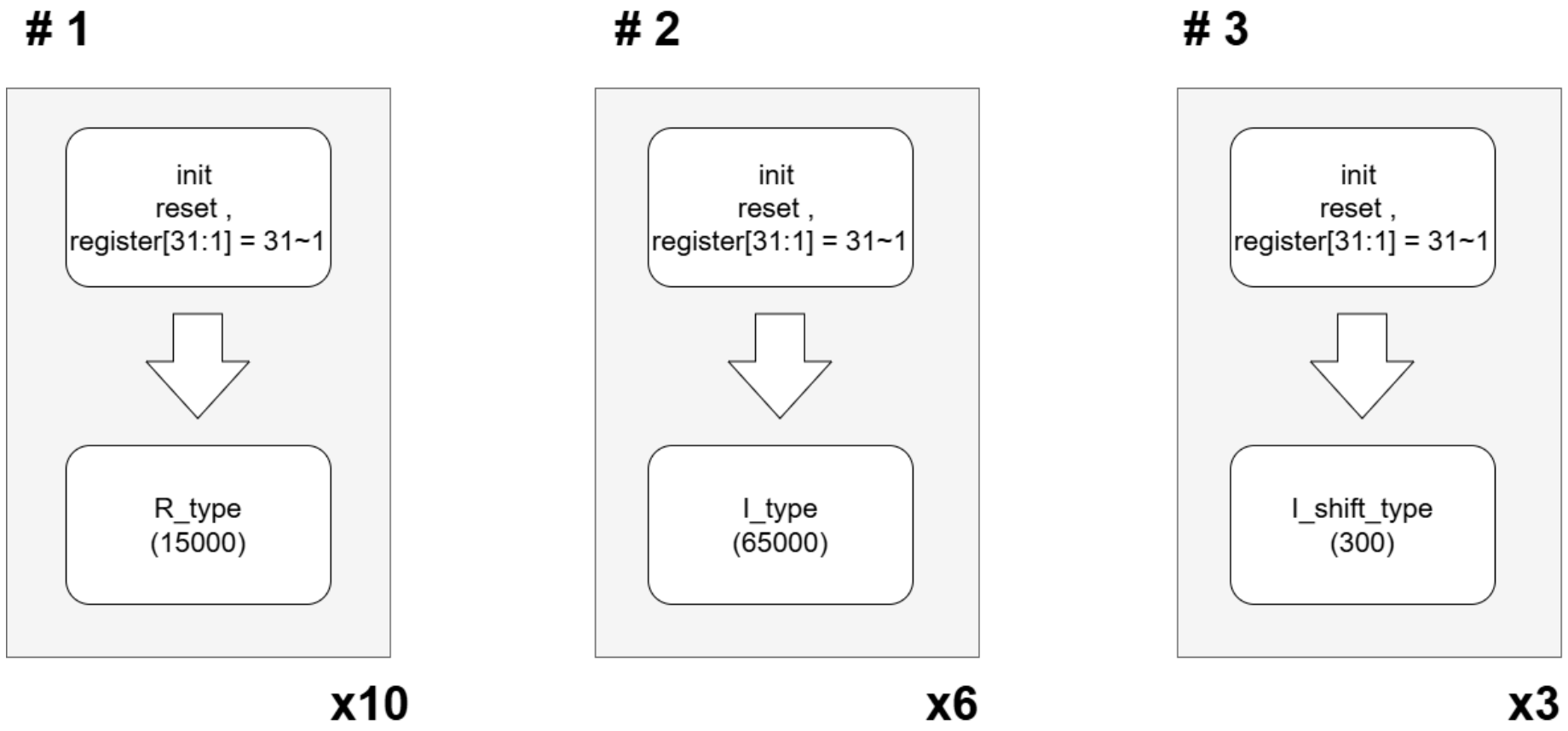
- R command x rs1 x rs2 = 10 x 1024
- I command x imm = 6 x 4096
- I shift command x imm(0~31) = 3 x 32
- R command x rd = 10 x 32
- I command x rd = 6 x 32
- I shift command x rd = 3 x 32



05. UVM Verification

■ 시나리오

필요한 평균 횟수 : R (1024) = $1024(\frac{1}{1024}+\frac{1}{1023}+\frac{1}{1022}+\ldots+1)$ ≈ 7,689, I (4096) ≈ 36,434, I_shift (32) ≈ 130



05. UVM Verification

■ 결과

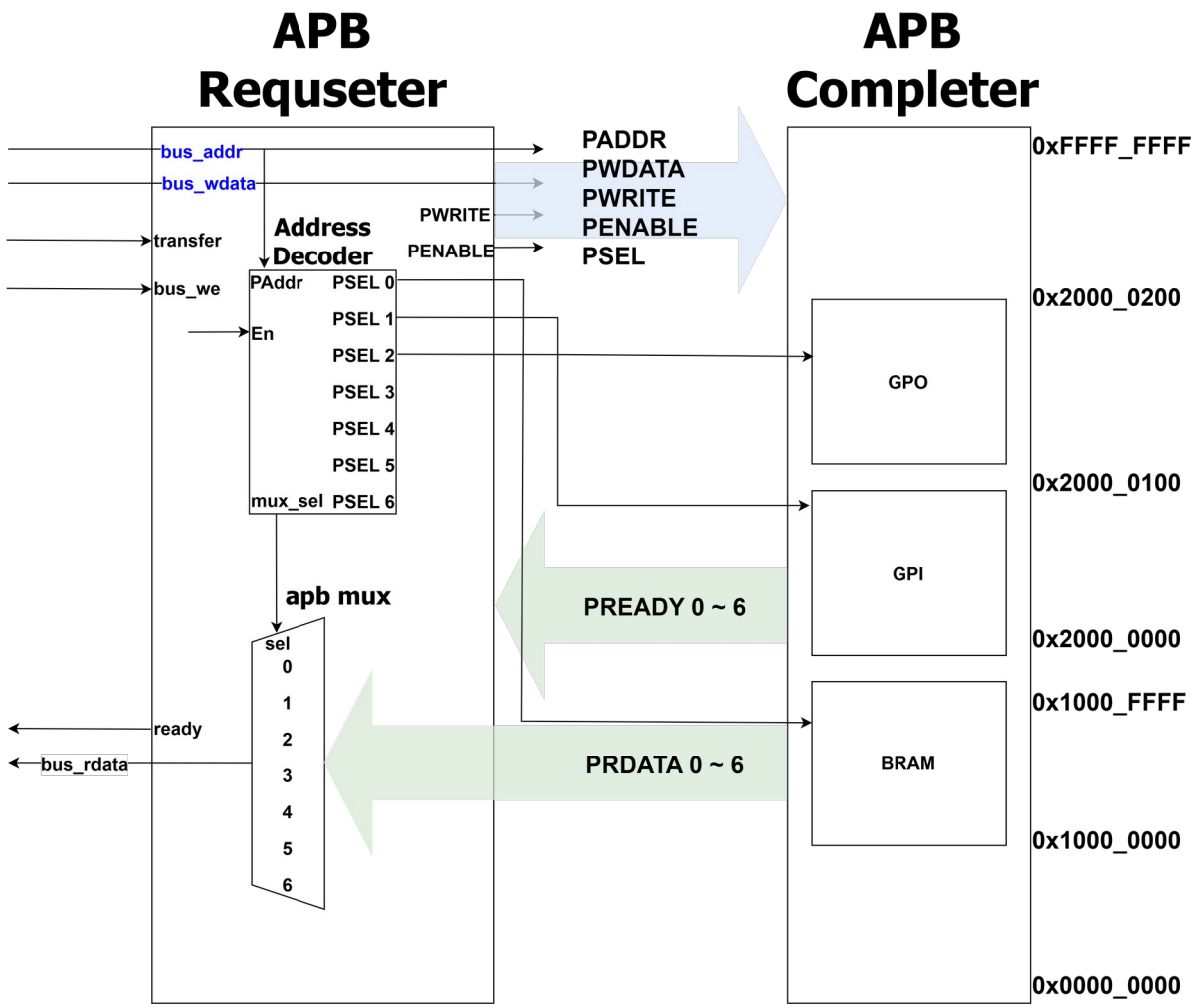
- 각 명령어 전 레지스터 초기화 : 19 x 31 = 589
- (15,000 x 9) + (65,000 x 6) + (300 x 3) + 589 = 541,489

```
[COV] ************************** rv32i_cpu Coverage Report **************************
[COV] Overall = 100.0% Reset = 100.0%
[COV] R commands = 100.0% I commands = 100.0% I shift commands = 100.0%
[COV] RS1 = 100.0% RS2 = 100.0% RD = 100.0%
[COV] R command x rs1 x rs2 = 100.0%
[COV] I command x imm = 100.0% I shift command x imm = 100.0%
[COV] R command x rd = 100.0%, I command x rd = 100.0%, I shift command x rd = 100.0%
[COV] ********************************************************************************
[SCB] PASS = 541489 FAIL = 0
```

Group	Score	Instances	U+C	U	C	X	Goal	Weight	AtLeast	PerInst	Overlap	AutoBinl	Missing	Comme
$unit::cpu_coverage::cp...	100.00%	100.00%	39765	0	39765	0	100%	1	1	1	1	64	64	
cp_i_command	100.00%		6	0	6	0	100%	1	1		1	0		
cp_i_shift_command	100.00%		3	0	3	0	100%	1	1		1	0		
cp_i_shift_imm	100.00%		32	0	32	0	100%	1	1		1	0		
cp_imm	100.00%		4096	0	4096	0	100%	1	1		1	0		
cp_r_command	100.00%		10	0	10	0	100%	1	1		1	0		
cp_rd	100.00%		32	0	32	0	100%	1	1		1	0		
cp_reset	100.00%		2	0	2	0	100%	1	1		1	0		
cp_rs1	100.00%		32	0	32	0	100%	1	1		1	0		
cp_rs2	100.00%		32	0	32	0	100%	1	1		1	0		
cx_i_imm	100.00%		24576	0	24576	0	100%	1	1				0	
cx_i_rd	100.00%		192	0	192	0	100%	1	1				0	
cx_i_shift_imm	100.00%		96	0	96	0	100%	1	1				0	
cx_i_shift_rd	100.00%		96	0	96	0	100%	1	1				0	
cx_r_rd	100.00%		320	0	320	0	100%	1	1				0	
cx_r_rs1_rs2	100.00%		10240	0	10240	0	100%	1	1				0	

06. APB Requester & Completer

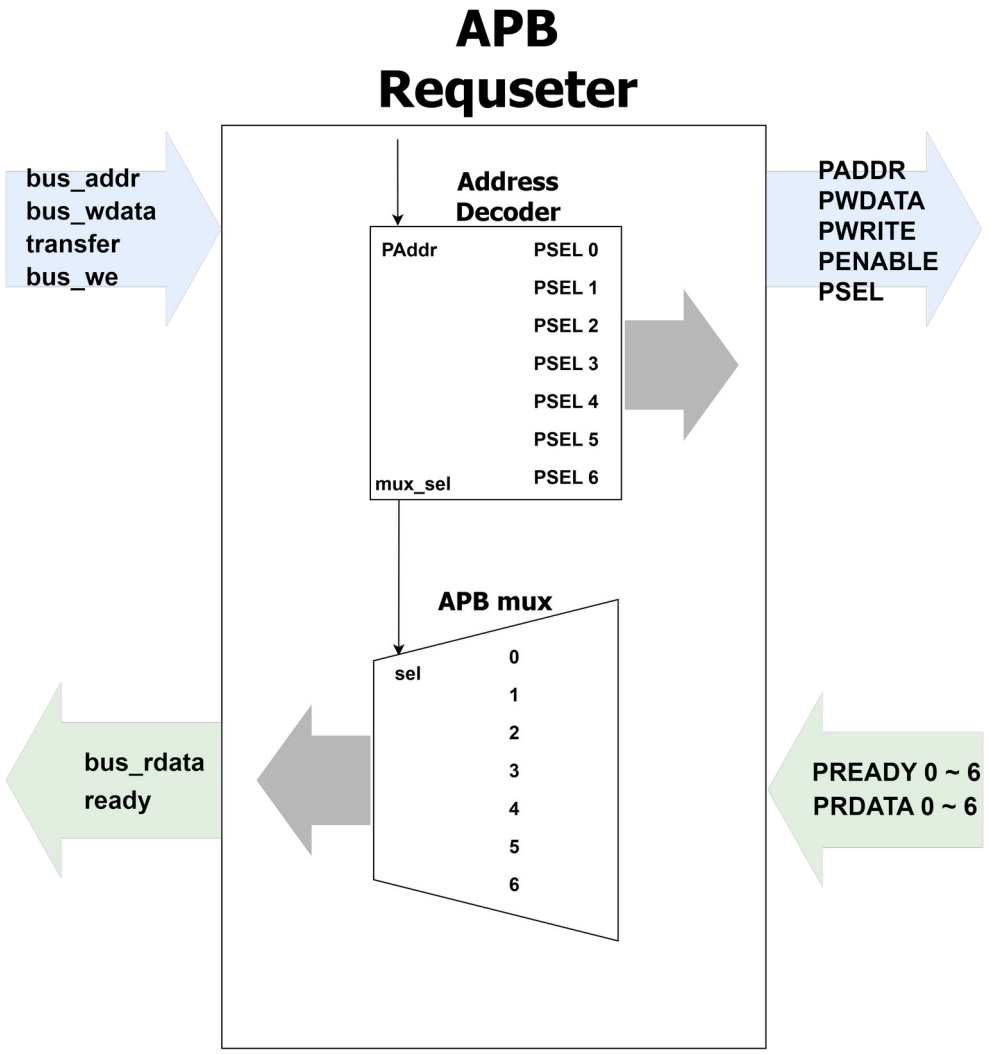
APB Requester& Completer



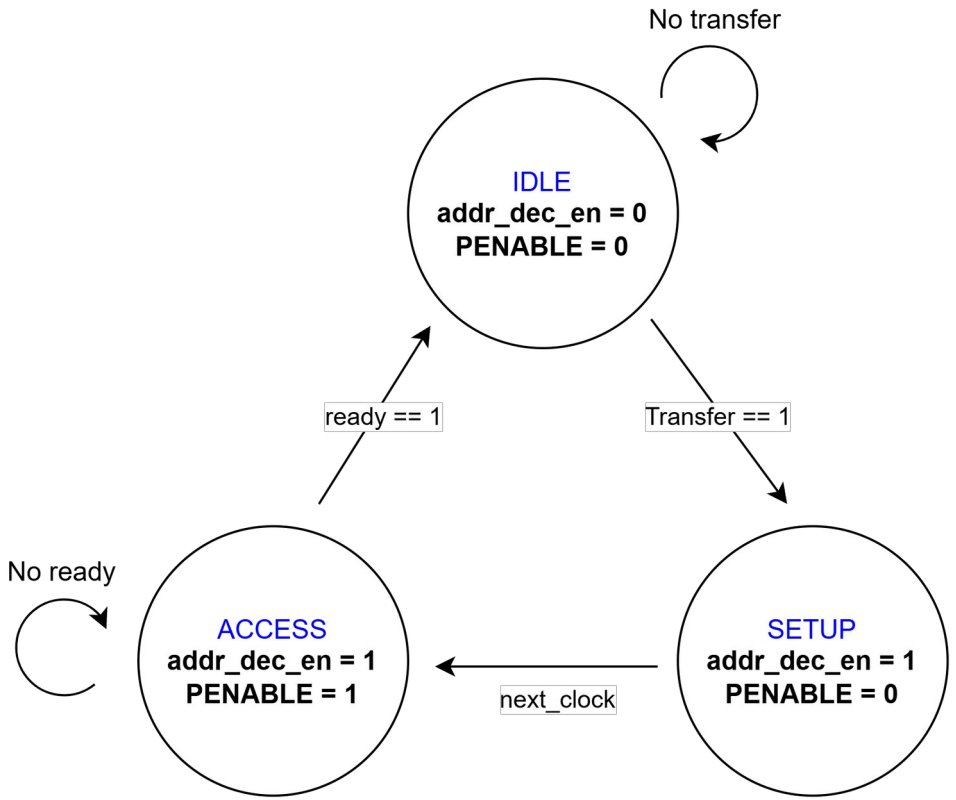
Requester	CPU bus 신호 (bus_addr, bus_wdata, transfer, bus_we) → APB 프로토콜 변환
Address Decoder	주소 → PSEL0~6 선택
Mux	선택된 Completer 의 PREADY/PRDATA 만 CPU 로 전송
Completer	APB 프로토콜에 따라 응답
요청 신호	(Requester → Completer) PADDR, PWDATA, PWRITE, PENABLE, PSEL
응답 신호	(Completer → Requester) PREADY, PRDATA
Memory Map	BRAM : 0x1000_0000 ~ 0x1000_FFFF

06. APB Requester & Completer

APB Requseter



FSM

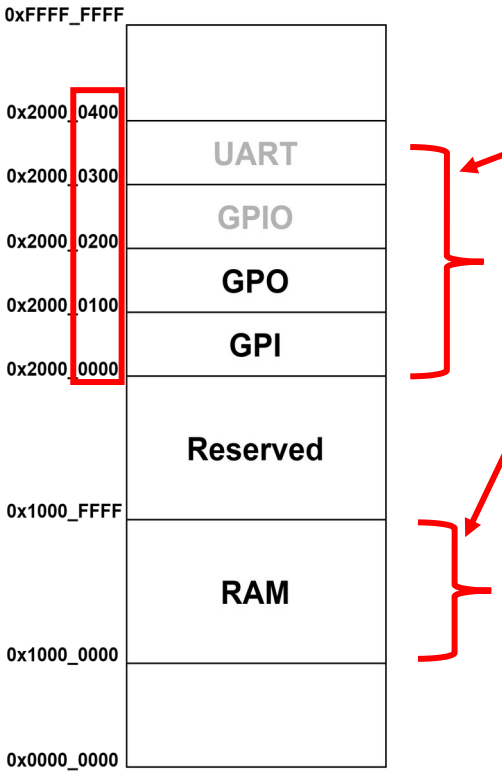


06. APB Requester & Completer

■ Address Decoder

상위 4 비트 판단 : RAM or Peripheral

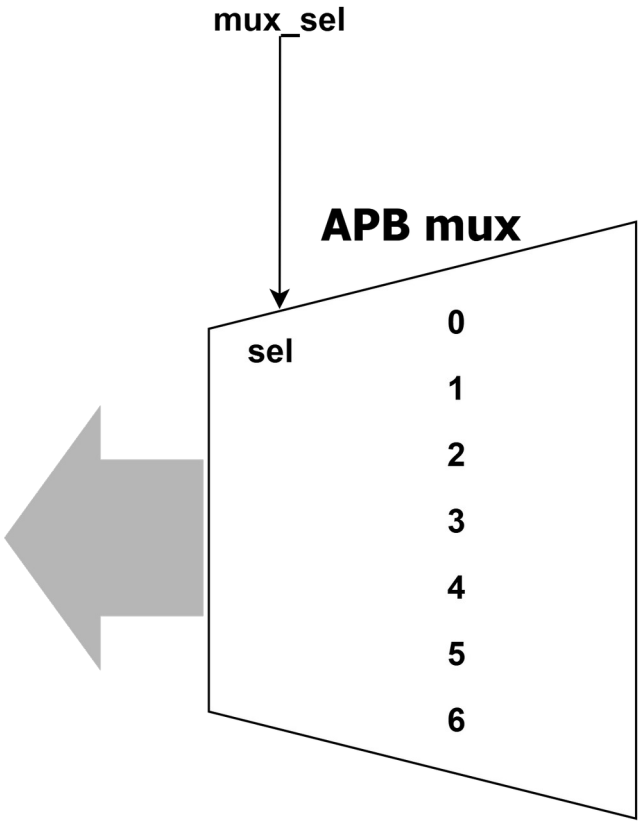
→ 비교 로직 단순화




```
if (enable)
    case (PADDR[31:28])
        4'b0001: begin  // APB
            PSEL0   = 1'b1;
            mux_sel = 3'b000;

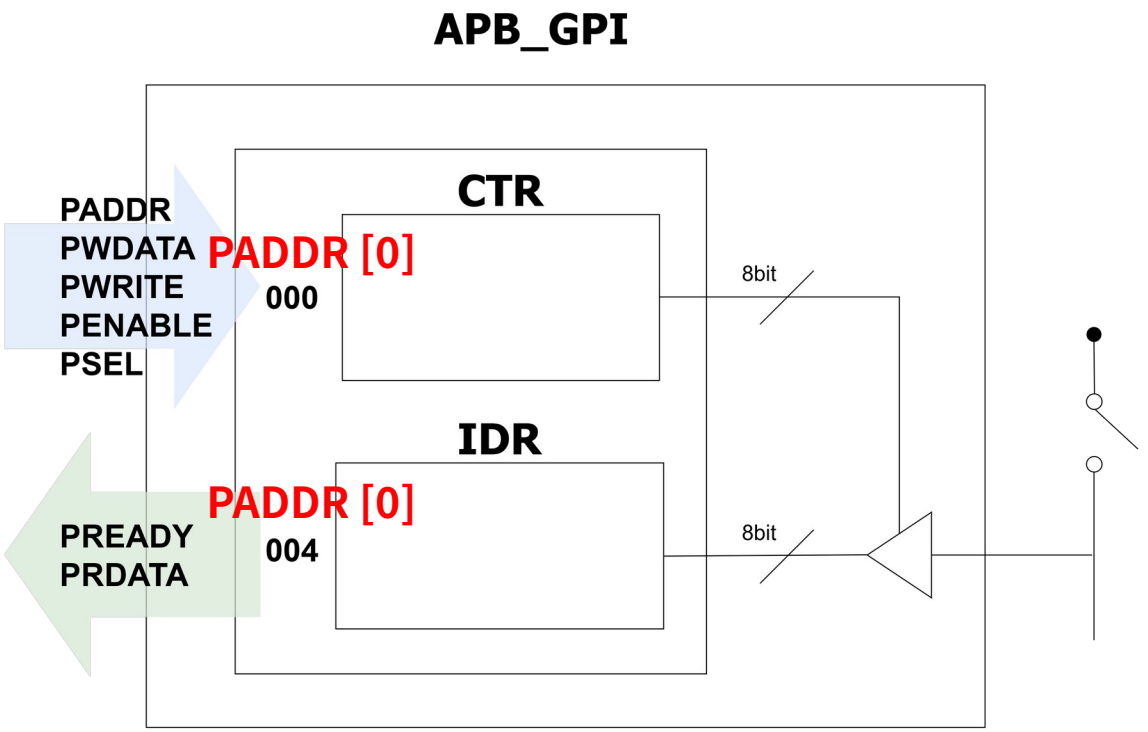
        end
        4'b0010: begin  // Per
            case (PADDR[11:8])
```

■ APB Mux



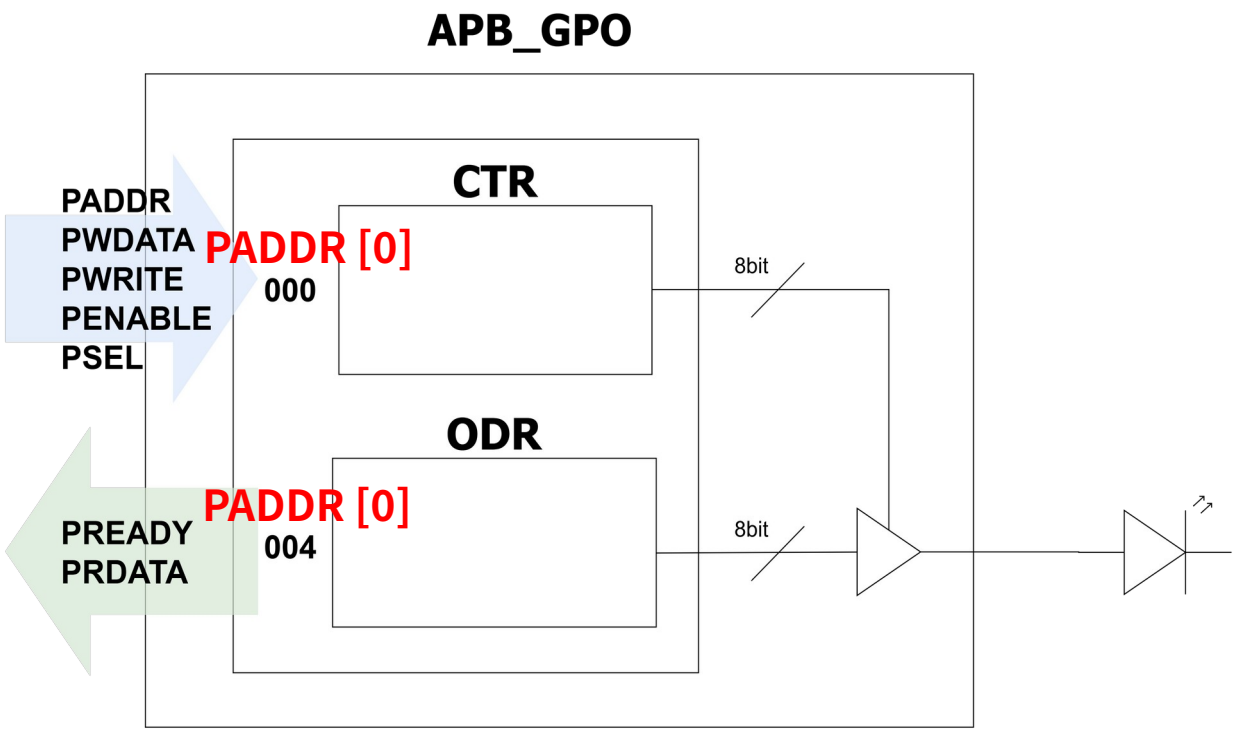
06. APB Requester & Completer

■ APB_GPI



CTR : 입력받을 스위치 활성화
IDR : 스위치 값 반영

■ APB_GPO



CTR : 출력보낼 LED 활성화
ODR : 출력할 LED
LED = CTR & ODR

06. APB Requester & Completer

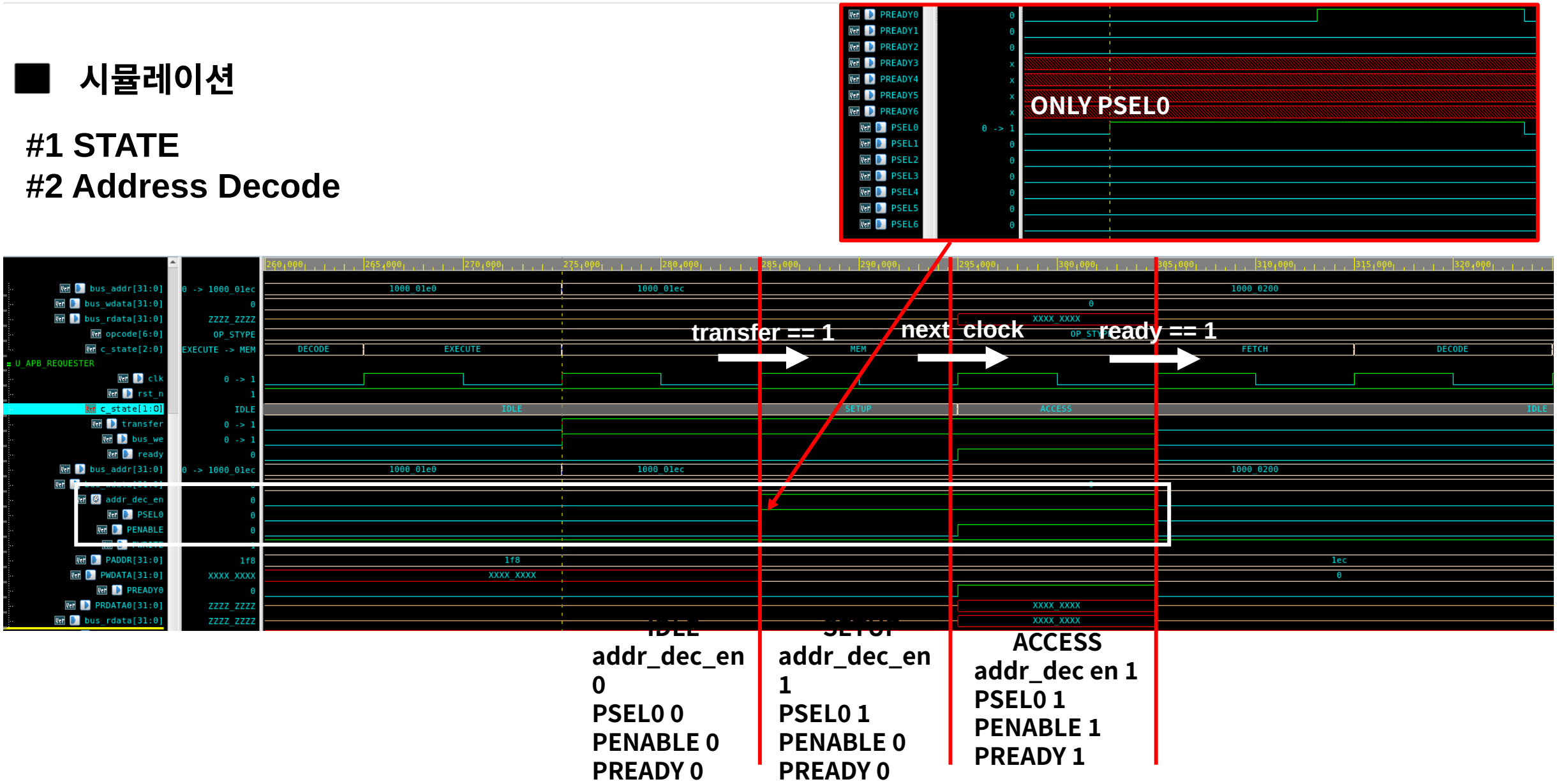
■ 시뮬레이션 시나리오

검증 항목		신호	예상 동작
APB Requester			
1	STATE	PSEL0, PENABLE, ready	PSEL0 → PENABLE → ready 순서로 1
2	Address Decode	PSEL0 ~ PSEL6	BRAM : PSEL0
3	Address, Data HOLD	PADDR, PWDATA, PWRITE	다음 SETUP 전까지 값 유지
APB Completer (BRAM)			
4	BRAM Write	PWDATA, PWRITE, PREADY0	ACCESS 상태에서 PWRITE = 1 쓰기 , PENABLE = 1 이후 상승 엣지에 dmem 에 저장
5	BRAM Read	PADDR, PWDATA, PWRITE, PREADY	ACCESS 상태에서 PWRITE = 0 읽기 , dmem 값 PRDATA0, ready 와 함께 bus_rdata 로 전달

06. APB Requester & Completer

시뮬레이션

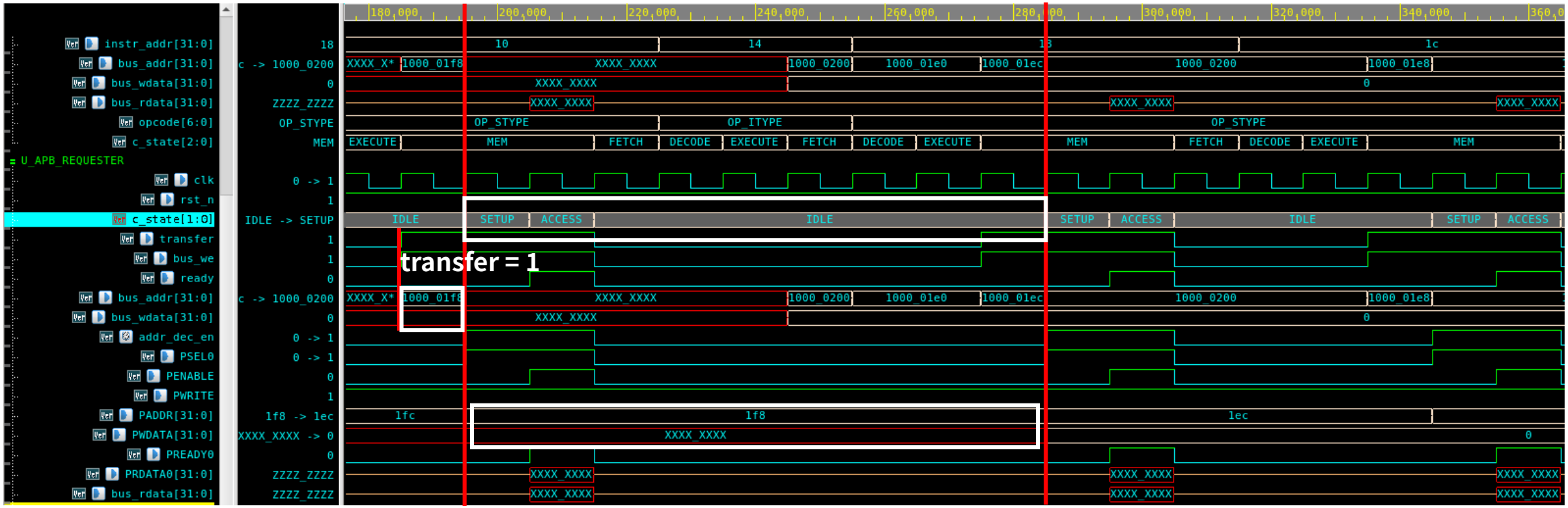
#1 STATE
#2 Address Decode



06. APB Requester & Completer

■ 시뮬레이션

#3 Address, Data HOLD



다음 SETUP 까지
PADDR, PWDATA 유지

06. APB Requester & Completer

■ 시뮬레이션

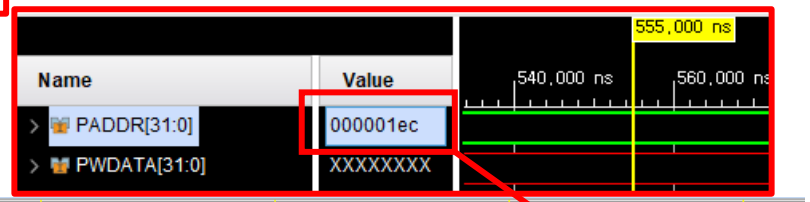
#4 BRAM Write
#5 BRAM Read

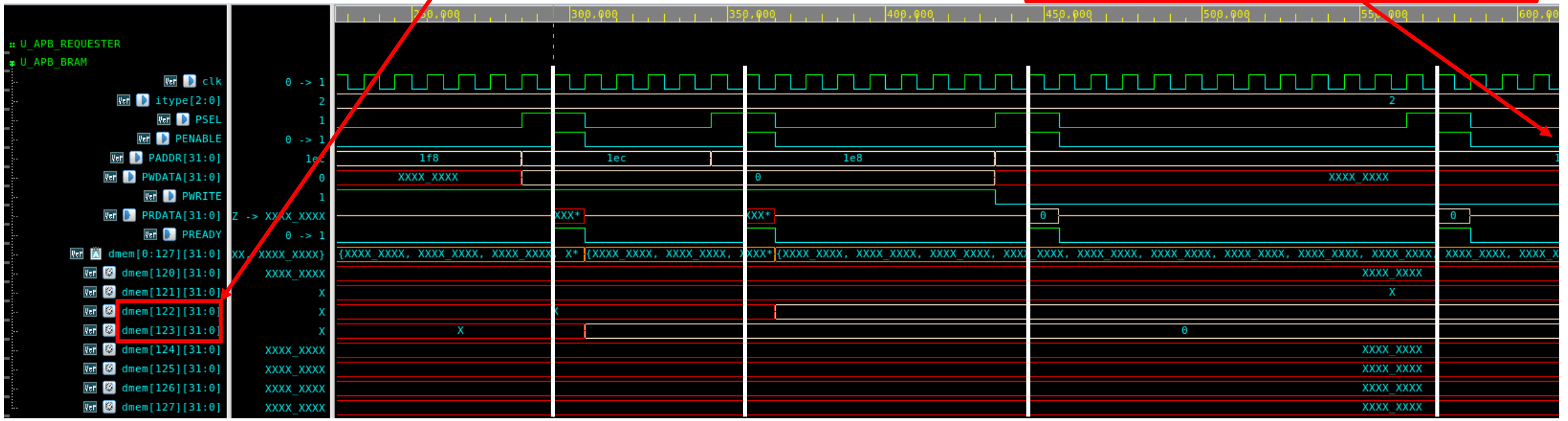
PADDR : H'1EC = D'488

byte address
D'492 /4 = D'123

PADDR : H'1E8 = D'492

byte address
D'492 /4 = D'122





PWRITE = 1
dmem 쓰기

PWRITE = 1
dmem 쓰기

PWRITE = 0
dmem 읽기

PWRITE = 0
dmem 읽기

06. APB Requester & Completer

■ 시뮬레이션 시나리오

```
int sw = 0;
// setup GPI
*(unsigned int*) 0x20000000 = 0x000000ff;
sw = *(unsigned int *) 0x20000004;
// setup GPO
*(unsigned int*) 0x20000100 = 0x0000000f;
*(unsigned int*) 0x20000104 = 0x00000055;
```

GPI.CTR

GPI.IDR

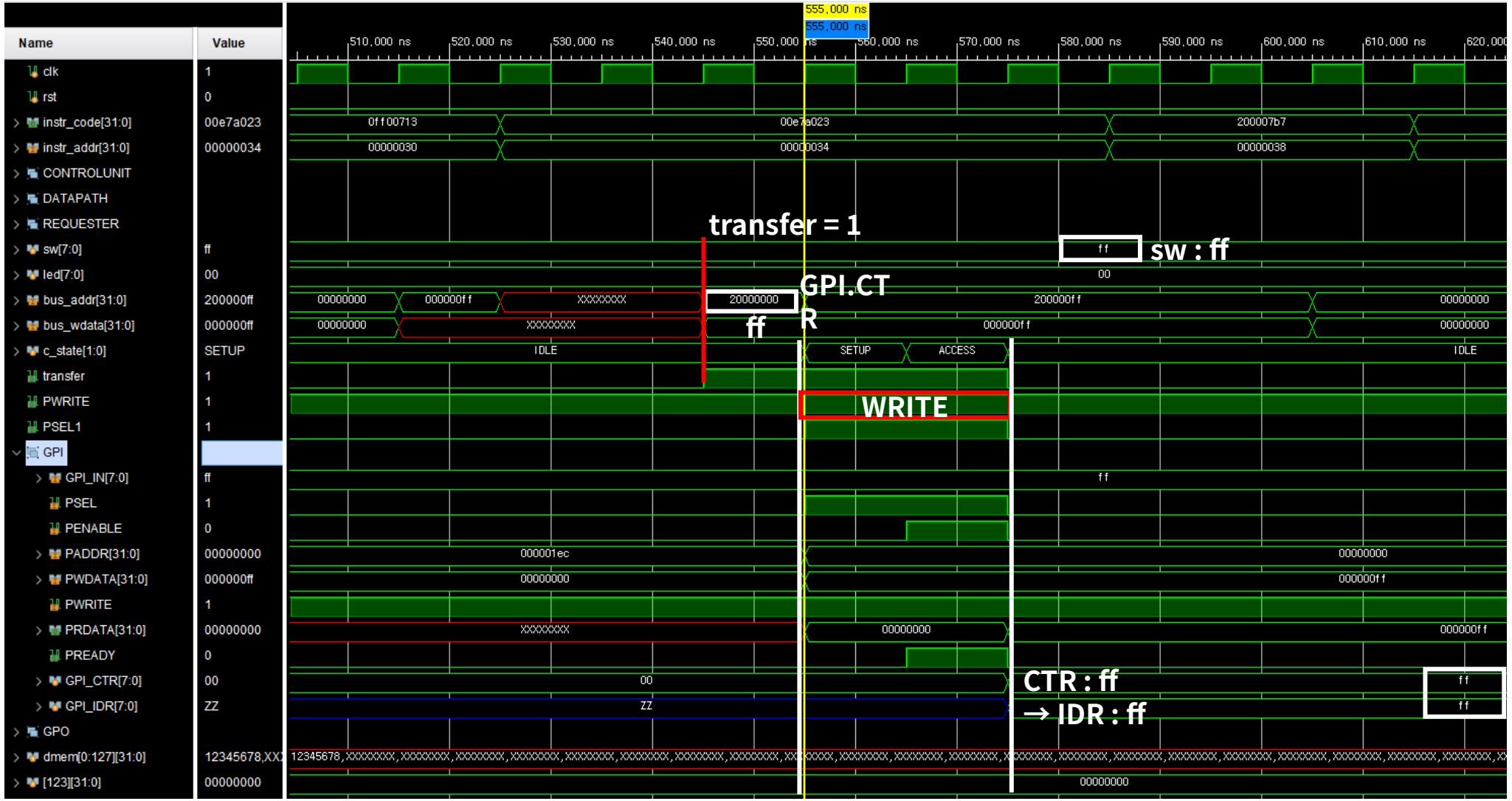
GPO.CTR

GPO.ODR

검증 항목		신호	예상 동작
APB GPI			
1	GPI CTR Write	GPI_CTR	ACCESS 상태 PWRITE=1 쓰기 , PENABLE=1 이후 상승 엣지에 GPI_CTR 에 저장
2	GPI IDR Read	PADDR, GPI_CTR, GPI_IN, PRDATA1, PREADY1	ACCESS 상태에서 PWRITE=0 읽기 , GPI_IDR 값 PREADY1 와 함께 PRDATA1 로 bus_rdata
3	BRAM Write	PADDR, PWDATA, PWRITE, PREADY0	ACCESS 상태 PWRITE=1 쓰기 , PENABLE=1 이후 GPI_IDR 을 읽은 값 (PWDATA) 이 dmem[123] 에 저장
APB GPO			
4	GPO CTR/ODR Write	PADDR[2], PWDATA, PWRITE, PREADY2	ACCESS 상태 PWRITE=1 쓰기 , PADDR[2]=0 이면 GPO_CTR, =1 이면 GPO_ODR 저장
5	led Output	GPO_CTR, GPO_ODR, led	GPO_CTR, GPO_ODR 갱신 led = GPO_ODR[7:0] & GPO_CTR[7:0]

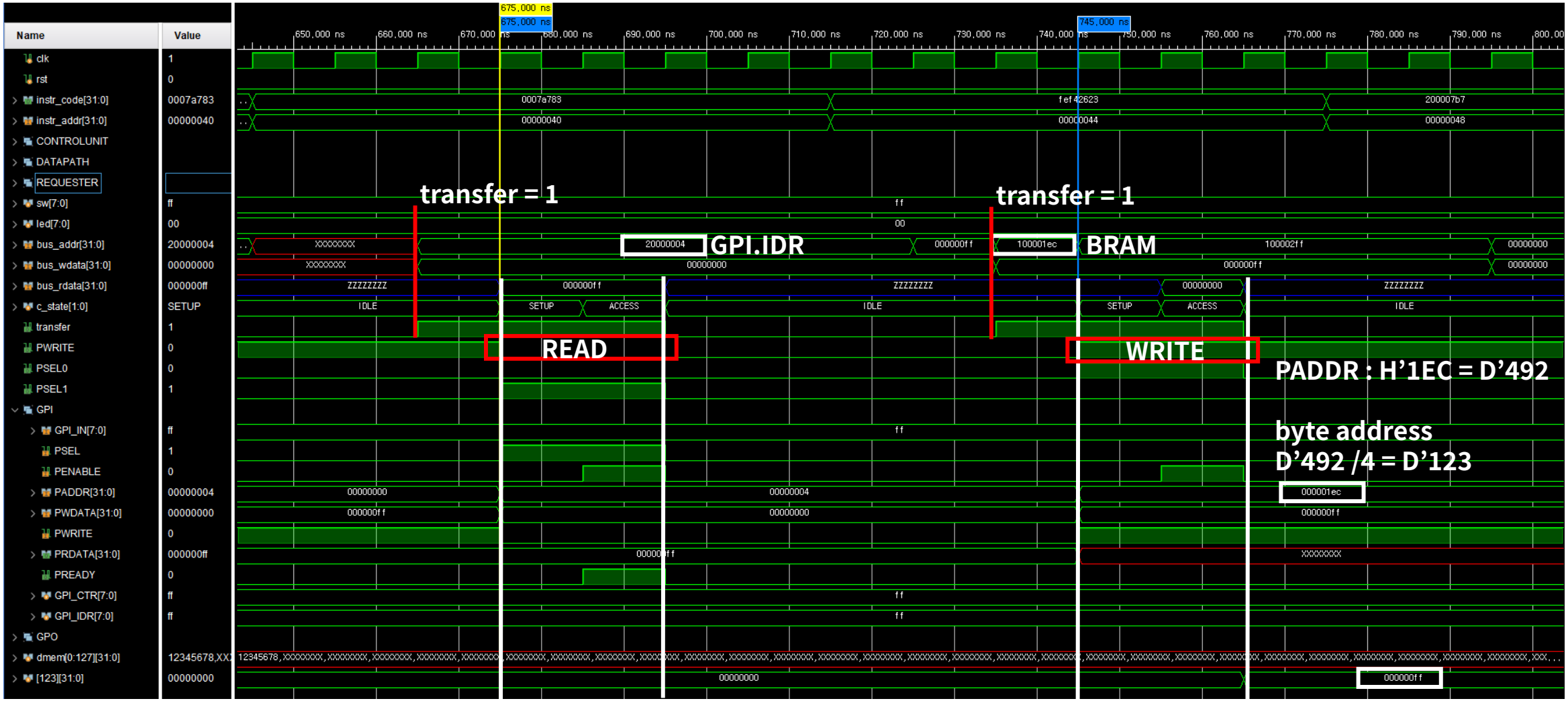
06. APB Requester & Completer

시뮬레이션 #1 GPI CTR Write



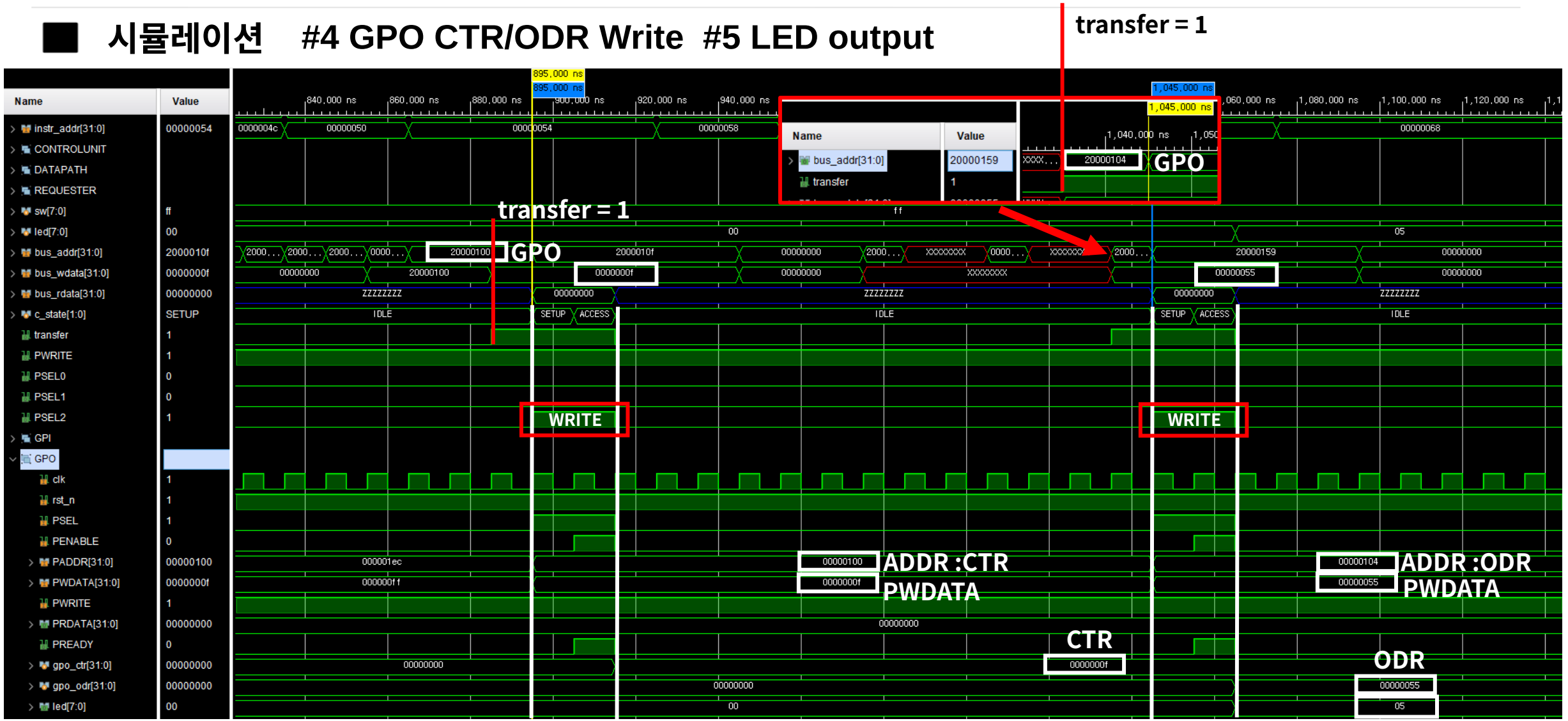
06. APB Requester & Completer

시뮬레이션 #2 GPI CTR Read #3 BRAM Write



06. APB Requester & Completer

시뮬레이션 #4 GPO CTR/ODR Write #5 LED output

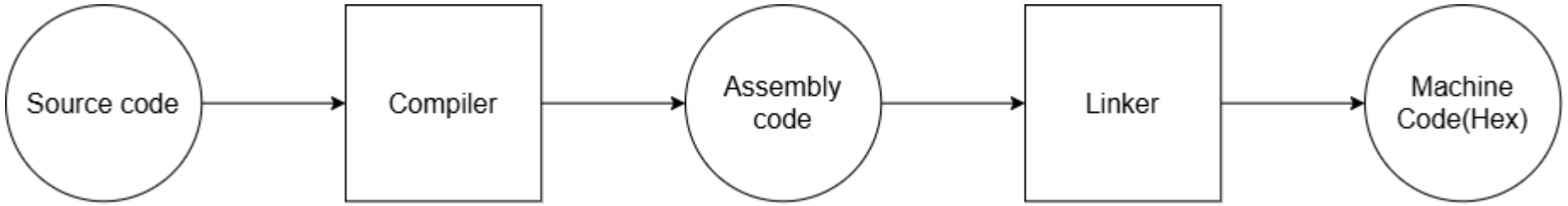


07. SW Verification

■ 개념

- CPU 는 High level 문법 인식 X

- High level 문법 → Machine Code



07. SW Verification

■ 시뮬레이션 시나리오

	검증 항목	신호	예상 동작
1	Jump	jal pc imm_ext	Pc+imm 값으로 jump
2	Bge	branch alu_control rs1, rs2	rs1, rs2 를 비교 후 조건에 따른 주소로 이동
3	a++	addi alu_result PADDR PWDATA	메모리의 a 값 read 후 1 을 증가시킨 값 메모리에 저장
4	Result	PADDR PWDATA PWRITE	10 회 반복 후 sum 주소에 누적합 저장
5	Loop	instr_addr pc pc_imm	while(1) 분기 주소에 진입 후 PC 가 동일 주소에 고정

```c
int adder(int a, int b);

void main(void){
    int a, sum;
    a = 0;
    sum = 0;

    while(a<10){
        a = a + 1;
        sum = adder(sum, a);
    }

    while(1);

    return;
}

int adder(int a, int b){
    return a + b;
}
```

1

```
main:
        addi    sp,sp,-32
        sw      ra,28(sp)
        sw      s0,24(sp)
        addi    s0,sp,32
        sw      zero,-20(s0)
        sw      zero,-24(s0)
        j       .L2
.L3:
        lw      a5,-20(s0)
        addi    a5,a5,1
        sw      a5,-20(s0)
        lw      a1,-20(s0)
        lw      a0,-24(s0)
        call    adder
        sw      a0,-24(s0)
.L2:
        lw      a4,-20(s0)
        li      a5,9
        ble     a4,a5,.L3
.L4:
        j       .L4
adder:
        addi    sp,sp,-32
        sw      ra,28(sp)
        sw      s0,24(sp)
        addi    s0,sp,32
        sw      a0,-20(s0)
        sw      a1,-24(s0)
        lw      a4,-20(s0)
        lw      a5,-24(s0)
        add     a5,a4,a5
        mv      a0,a5
        lw      ra,28(sp)
        lw      s0,24(sp)
        addi    sp,sp,32
        jr      ra
```

07. SW Verification

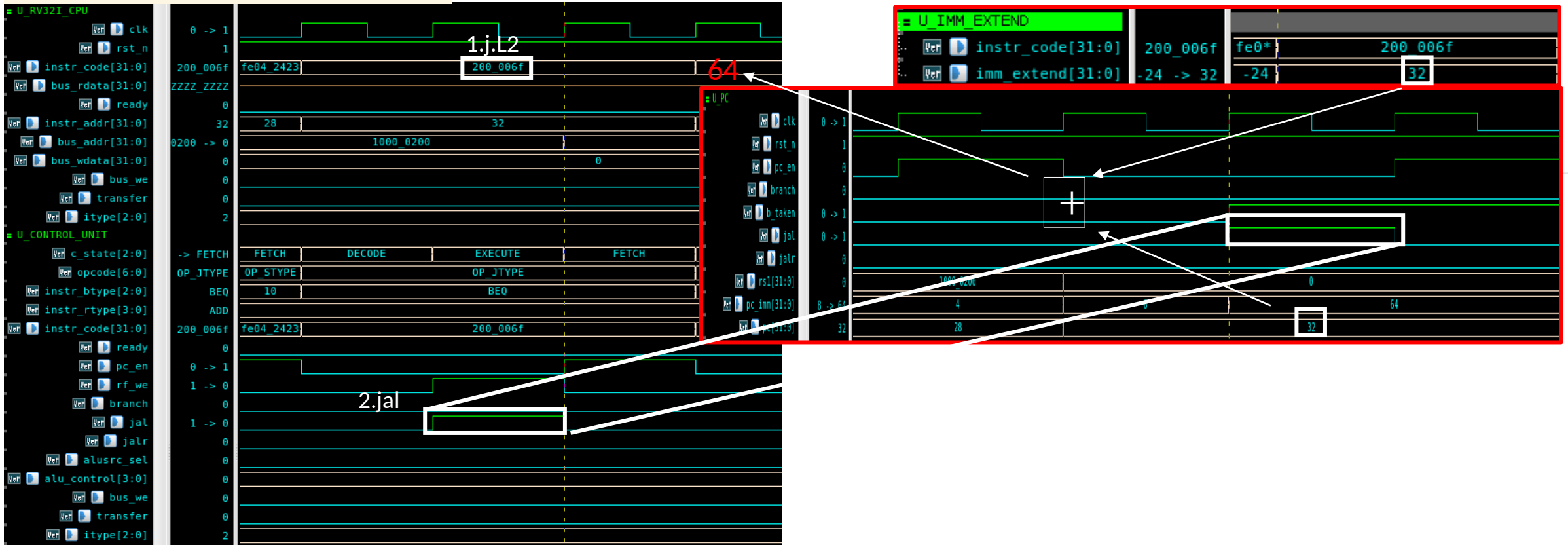
시뮬레이션 - Jump

```
Assembly =   jal x0, 32

Binary =     0000 0010 0000 0000 0000 0000 0110 1111

Hexadecimal = 0x0200006f

Format =     J-type
```



07. SW Verification

■ 시뮬레이션 시나리오

	검증 항목	신호	예상 동작
1	Jump	jal pc imm_ext	Pc+imm 값으로 jump
2	Bge	branch alu_control rs1, rs2	rs1, rs2 를 비교 후 조건에 따른 주소로 이동
3	a++	addi alu_result PADDR PWDATA	메모리의 a 값 read 후 1 을 증가시킨 값 메모리에 저장
4	Result	PADDR PWDATA PWRITE	10 회 반복 후 sum 주소에 누적합 저장
5	Loop	instr_addr pc pc_imm	while(1) 분기 주소에 진입 후 PC 가 동일 주소에 고정

```c
int adder(int a, int b);

void main(void){
    int a, sum;
    a = 0;
    sum = 0;

    while(a<10){
        a = a + 1;
        sum = adder(sum, a);
    }

    while(1);

    return;
}

int adder(int a, int b){
    return a + b;
}
```

2

```
main:
        addi    sp,sp,-32
        sw      ra,28(sp)
        sw      s0,24(sp)
        addi    s0,sp,32
        sw      zero,-20(s0)
        sw      zero,-24(s0)
        j       .L2
.L3:
        lw      a5,-20(s0)
        addi    a5,a5,1
        sw      a5,-20(s0)
        lw      a1,-20(s0)
        lw      a0,-24(s0)
        call    adder
        sw      a0,-24(s0)
.L2:
        lw      a4,-20(s0)
        li      a5,9
        ble     a4,a5,.L3
.L4:
        j       .L4
adder:
        addi    sp,sp,-32
        sw      ra,28(sp)
        sw      s0,24(sp)
        addi    s0,sp,32
        sw      a0,-20(s0)
        sw      a1,-24(s0)
        lw      a4,-20(s0)
        lw      a5,-24(s0)
        add     a5,a4,a5
        mv      a0,a5
        lw      ra,28(sp)
        lw      s0,24(sp)
        addi    sp,sp,32
        jr      ra
```

07. SW Verification

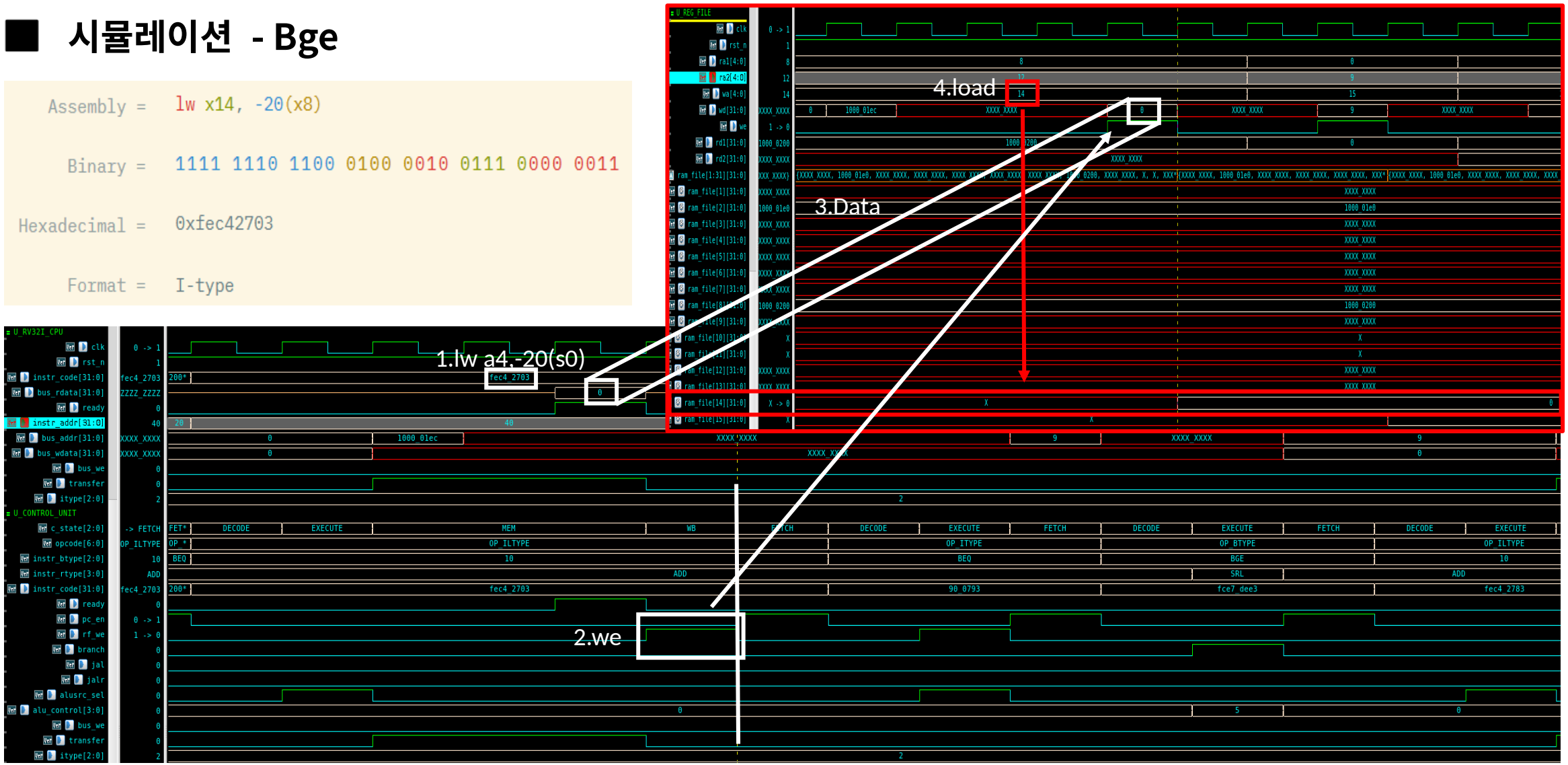
시뮬레이션 - Bge

```
Assembly =    lw x14, -20(x8)

  Binary =    1111 1110 1100 0100 0010 0111 0000 0011

Hexadecimal = 0xfec42703

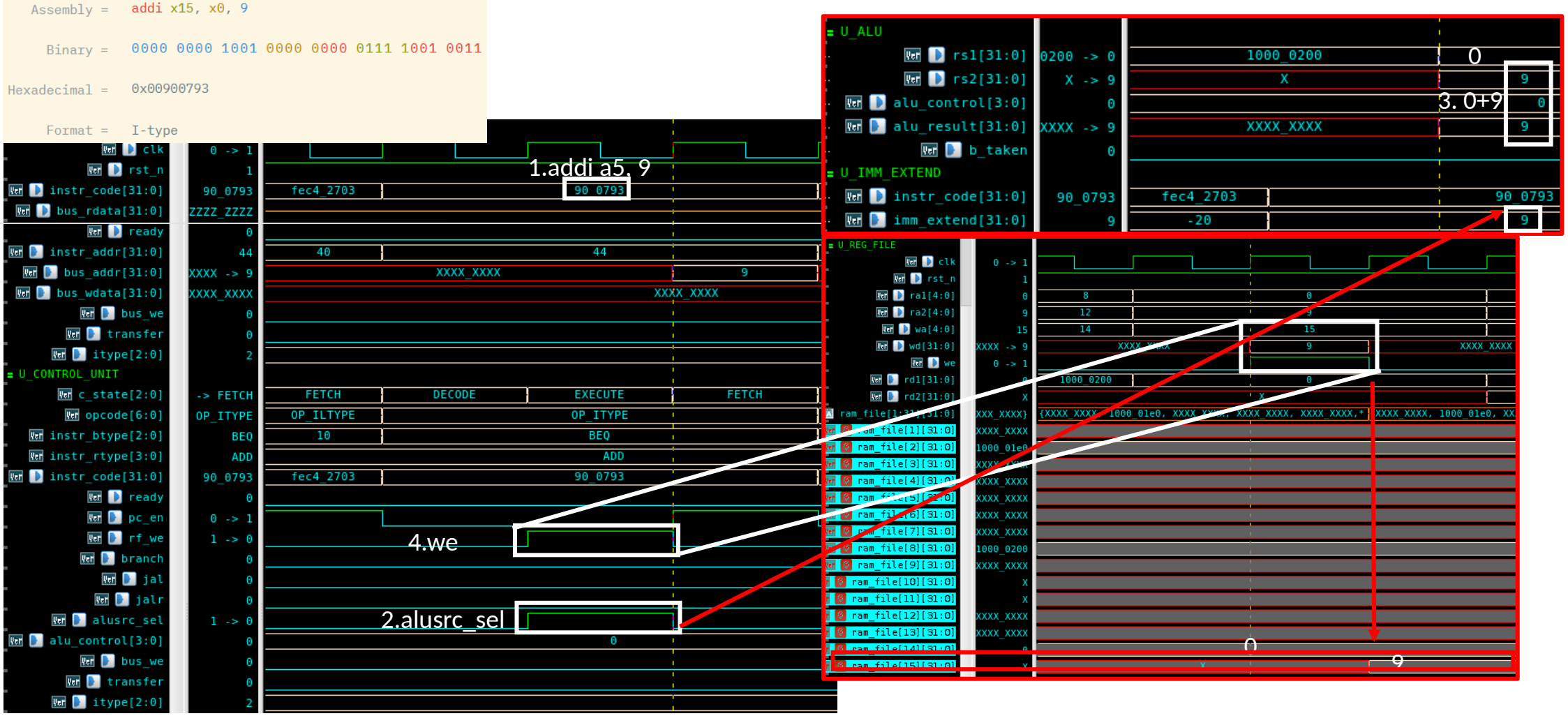
  Format =    I-type
```



07. SW Verification

시뮬레이션 - Bge

```
Assembly =   addi x15, x0, 9
  Binary =   0000 0000 1001 0000 0000 0111 1001 0011
Hexadecimal = 0x00900793
  Format =   I-type
```



07. SW Verification

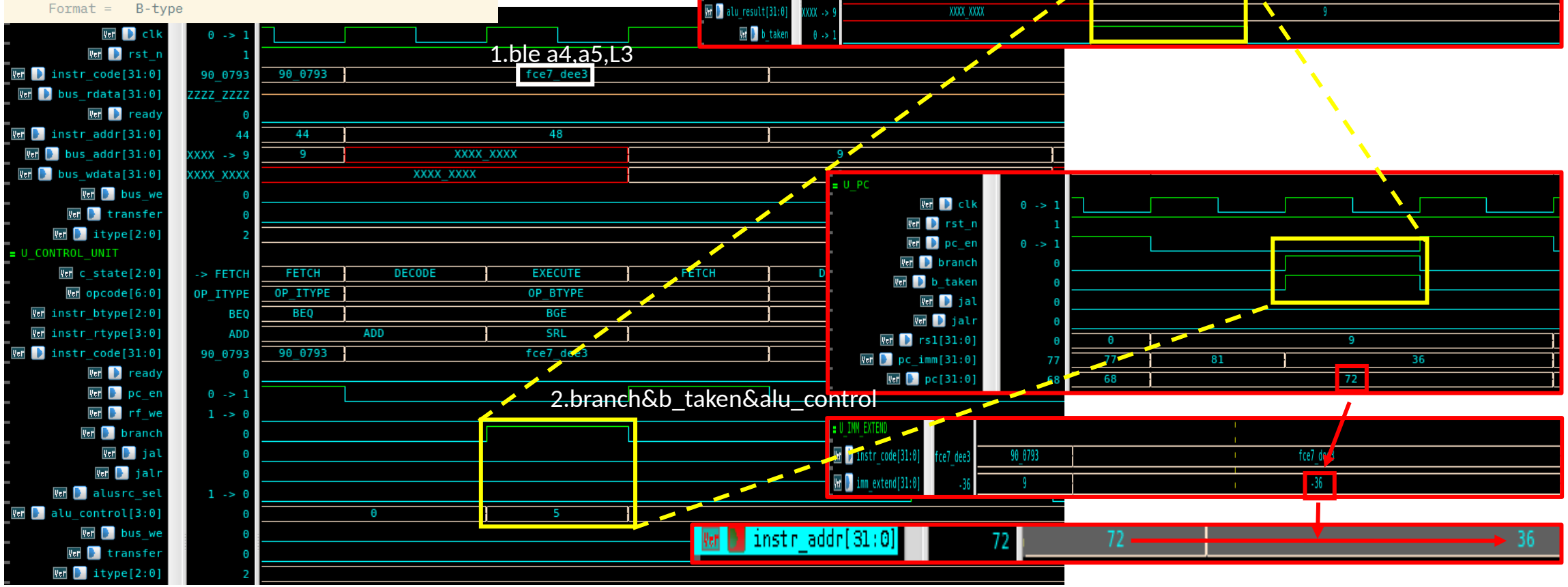
시뮬레이션 - Bge

```
Assembly =   bge x15, x14, -36

  Binary =   1111 1100 1110 0111 1101 1110 1110 0011

Hexadecimal = 0xfce7dee3

  Format =   B-type
```



07. SW Verification

■ 시뮬레이션 시나리오

	검증 항목	신호	예상 동작
1	Jump	jal pc imm_ext	Pc+imm 값으로 jump
2	Bge	branch alu_control rs1, rs2	rs1, rs2 를 비교 후 조건에 따른 주소로 이동
3	a++	addi alu_result PADDR PWDATA	메모리의 a 값 read 후 1 을 증가시킨 값 메모리에 저장
4	Result	PADDR PWDATA PWRITE	10 회 반복 후 sum 주소에 누적합 저장
5	Loop	instr_addr pc pc_imm	while(1) 분기 주소에 진입 후 PC 가 동일 주소에 고정

```c
int adder(int a, int b);

void main(void){
    int a, sum;
    a = 0;
    sum = 0;

    while(a<10){
        a = a + 1;
        sum = adder(sum, a);
    }

    while(1);

    return;
}

int adder(int a, int b){
    return a + b;
}
```

3

```
main:
        addi    sp,sp,-32
        sw      ra,28(sp)
        sw      s0,24(sp)
        addi    s0,sp,32
        sw      zero,-20(s0)
        sw      zero,-24(s0)
        j       .L2
.L3:
        lw      a5,-20(s0)
        addi    a5,a5,1
        sw      a5,-20(s0)
        lw      a1,-20(s0)
        lw      a0,-24(s0)
        call    adder
        sw      a0,-24(s0)
.L2:
        lw      a4,-20(s0)
        li      a5,9
        ble     a4,a5,.L3
.L4:
        j       .L4
adder:
        addi    sp,sp,-32
        sw      ra,28(sp)
        sw      s0,24(sp)
        addi    s0,sp,32
        sw      a0,-20(s0)
        sw      a1,-24(s0)
        lw      a4,-20(s0)
        lw      a5,-24(s0)
        add     a5,a4,a5
        mv      a0,a5
        lw      ra,28(sp)
        lw      s0,24(sp)
        addi    sp,sp,32
        jr      ra
```

07. SW Verification

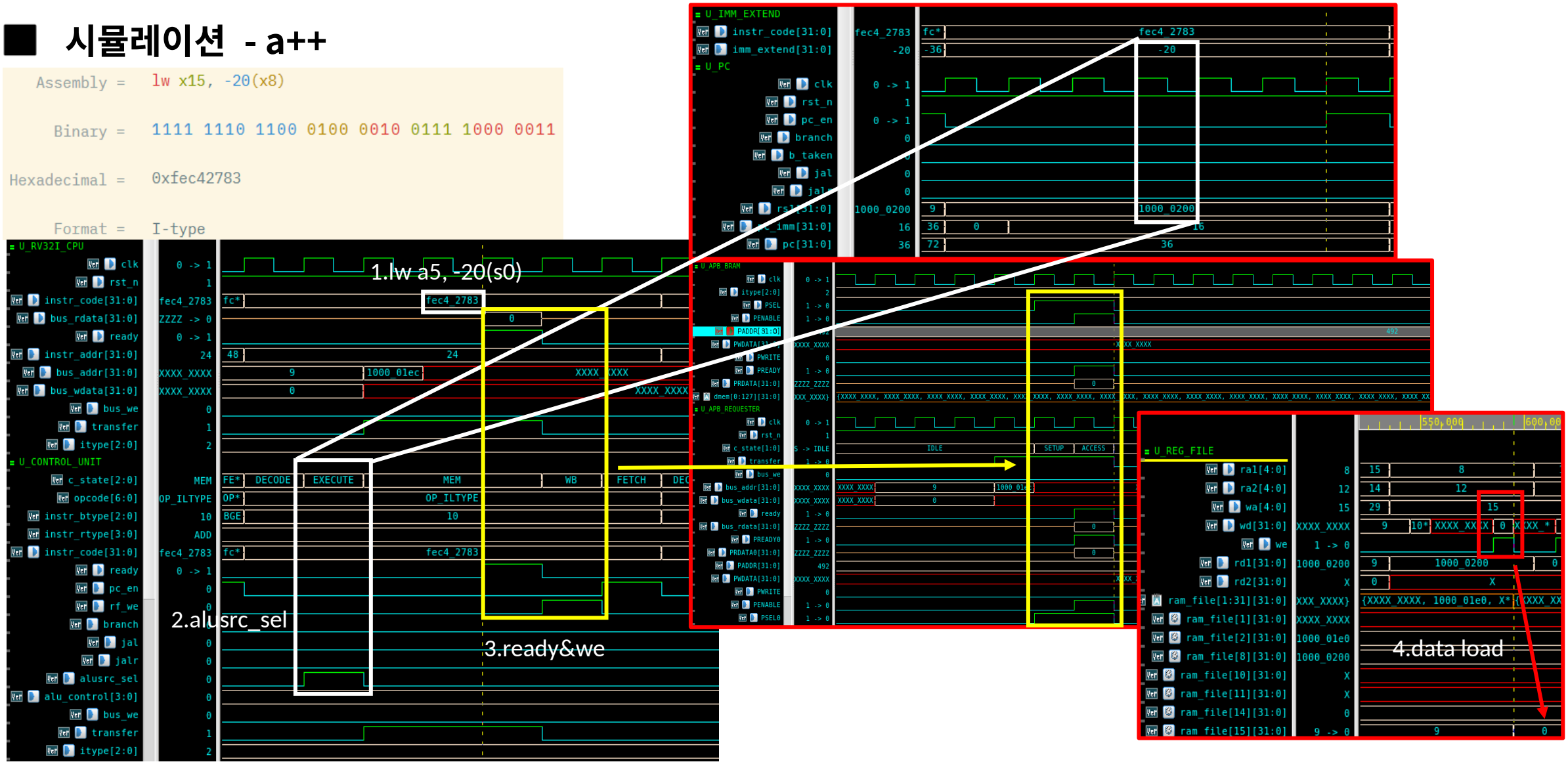
시뮬레이션 - a++

```
Assembly =   lw x15, -20(x8)

  Binary =   1111 1110 1100 0100 0010 0111 1000 0011

Hexadecimal = 0xfec42783

  Format =   I-type
```



07. SW Verification

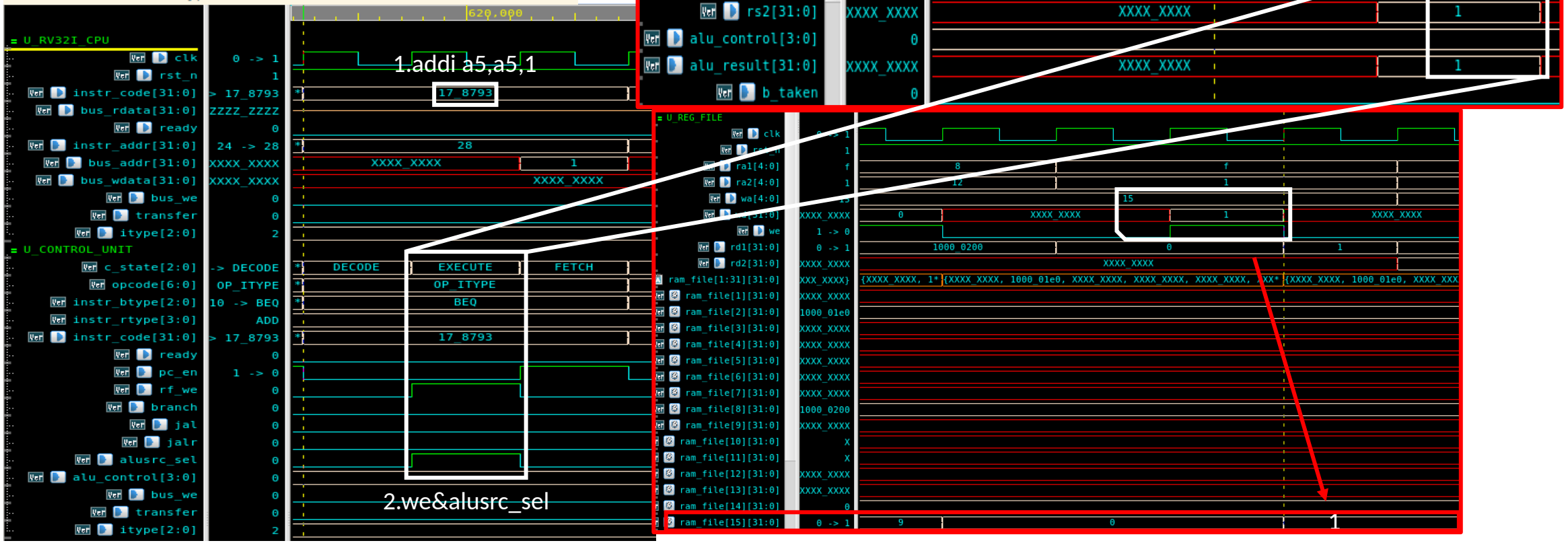
■ 시뮬레이션 - a++

```
Assembly =   addi x15, x15, 1

  Binary =   0000 0000 0001 0111 1000 0111 1001 0011

Hexadecimal = 0x00178793

  Format =   I-type
```



07. SW Verification

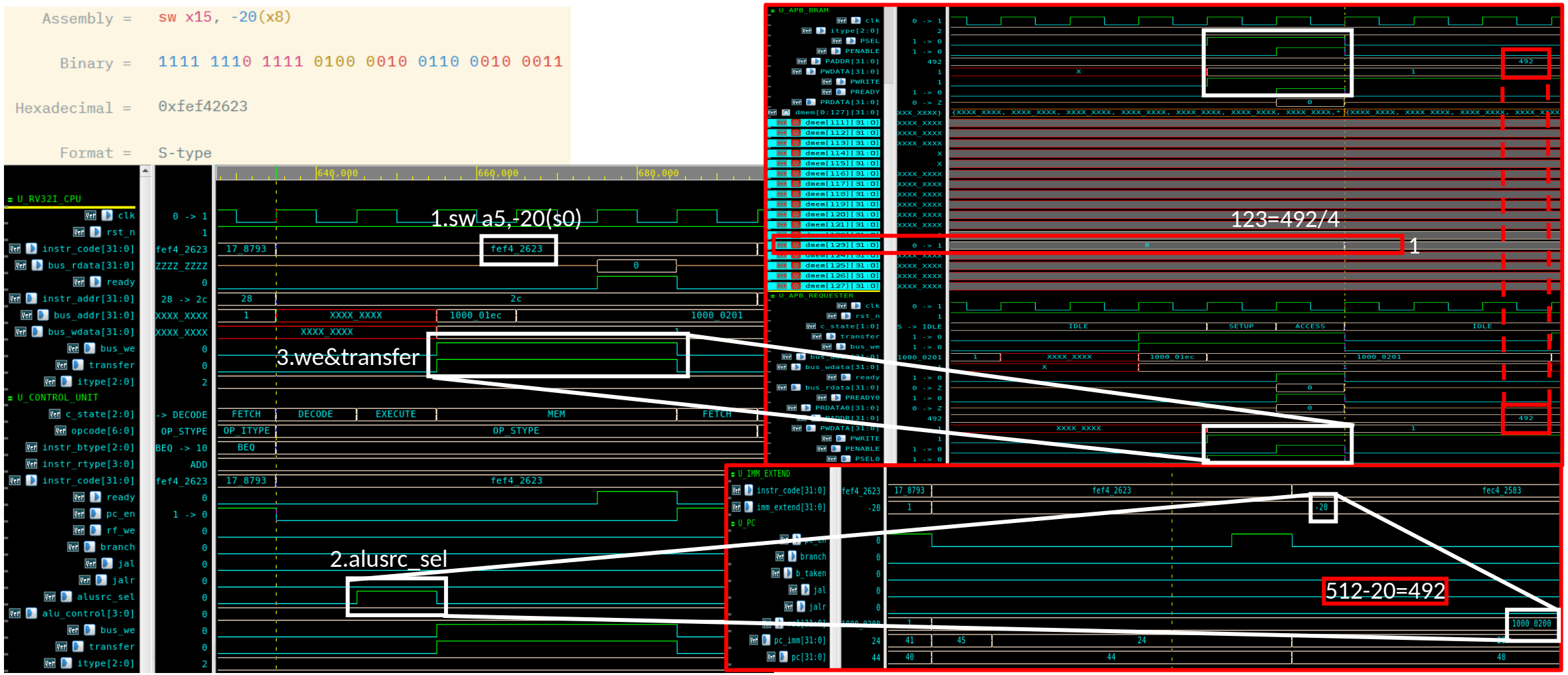
■ 시뮬레이션 - a++

```
Assembly =   sw x15, -20(x8)

  Binary =   1111 1110 1111 0100 0010 0110 0010 0011

Hexadecimal = 0xfef42623

  Format =   S-type
```



07. SW Verification

■ 시뮬레이션 시나리오

	검증 항목	신호	예상 동작
1	Jump	jal pc imm_ext	Pc+imm 값으로 jump
2	Bge	branch alu_control rs1, rs2	rs1, rs2 를 비교 후 조건에 따른 주소로 이동
3	a++	addi alu_result PADDR PWDATA	메모리의 a 값 read 후 1 을 증가시킨 값 메모리에 저장
4	Result	PADDR PWDATA PWRITE	10 회 반복 후 sum 주소에 누적합 저장
5	Loop	instr_addr pc pc_imm	while(1) 분기 주소에 진입 후 PC 가 동일 주소에 고정

```c
int adder(int a, int b);

void main(void){
    int a, sum;
    a = 0;
    sum = 0;

    while(a<10){
        a = a + 1;
        sum = adder(sum, a);
    }

    while(1);

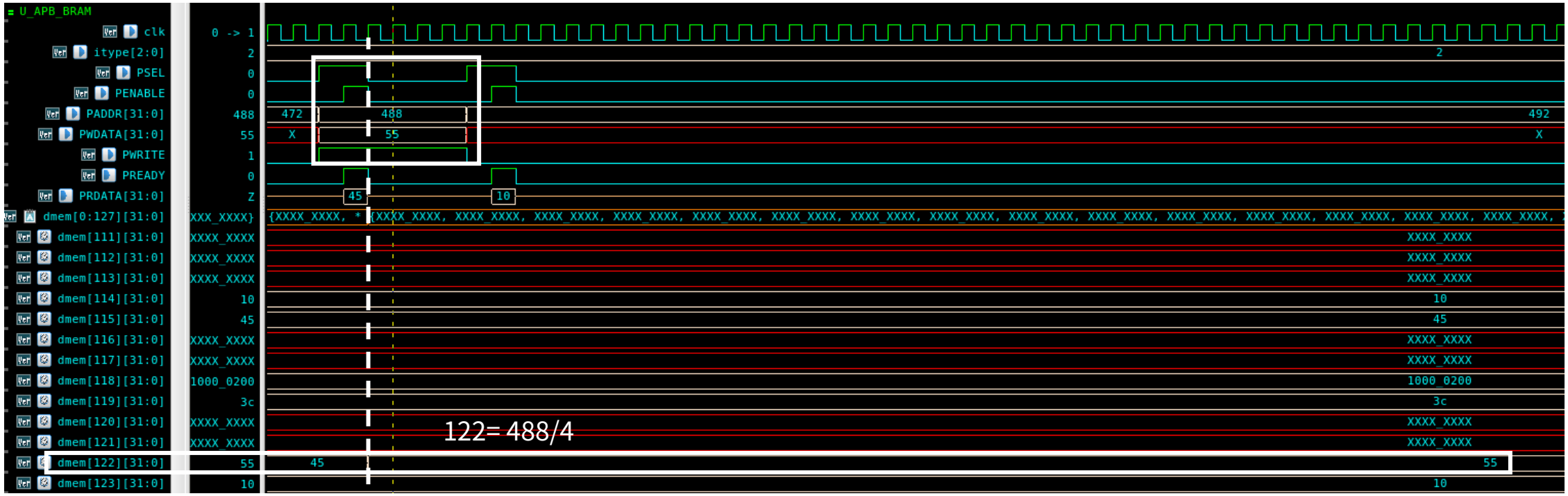
    return;
}

int adder(int a, int b){
    return a + b;
}
```

```
main:
        addi    sp,sp,-32
        sw      ra,28(sp)
        sw      s0,24(sp)
        addi    s0,sp,32
        sw      zero,-20(s0)
        sw      zero,-24(s0)
        j       .L2
.L3:
        lw      a5,-20(s0)
        addi    a5,a5,1
        sw      a5,-20(s0)
        lw      a1,-20(s0)
        lw      a0,-24(s0)
        call    adder
        sw      a0,-24(s0)
.L2:
        lw      a4,-20(s0)
        li      a5,9
        ble     a4,a5,.L3
.L4:
        j       .L4
adder:
        addi    sp,sp,-32
        sw      ra,28(sp)
        sw      s0,24(sp)
        addi    s0,sp,32
        sw      a0,-20(s0)
        sw      a1,-24(s0)
        lw      a4,-20(s0)
        lw      a5,-24(s0)
        add     a5,a4,a5
        mv      a0,a5
        lw      ra,28(sp)
        lw      s0,24(sp)
        addi    sp,sp,32
        jr      ra
```

07. SW Verification

■ 시뮬레이션 - Result



07. SW Verification

■ 시뮬레이션 시나리오

	검증 항목	신호	예상 동작
1	Jump	jal pc imm_ext	Pc+imm 값으로 jump
2	Bge	branch alu_control rs1, rs2	rs1, rs2 를 비교 후 조건에 따른 주소로 이동
3	a++	addi alu_result PADDR PWDATA	메모리의 a 값 read 후 1 을 증가시킨 값 메모리에 저장
4	Result	PADDR PWDATA PWRITE	10 회 반복 후 sum 주소에 누적합 저장
5	Loop	instr_addr pc pc_imm	while(1) 분기 주소에 진입 후 PC 가 동일 주소에 고정

```c
int adder(int a, int b);

void main(void){
    int a, sum;
    a = 0;
    sum = 0;

    while(a<10){
        a = a + 1;
        sum = adder(sum, a);
    }

    while(1);

    return;
}

int adder(int a, int b){
    return a + b;
}
```

```
main:
        addi    sp,sp,-32
        sw      ra,28(sp)
        sw      s0,24(sp)
        addi    s0,sp,32
        sw      zero,-20(s0)
        sw      zero,-24(s0)
        j       .L2
.L3:
        lw      a5,-20(s0)
        addi    a5,a5,1
        sw      a5,-20(s0)
        lw      a1,-20(s0)
        lw      a0,-24(s0)
        call    adder
        sw      a0,-24(s0)
.L2:
        lw      a4,-20(s0)
        li      a5,9
        ble     a4,a5,.L3
.L4:
        j       .L4
adder:
        addi    sp,sp,-32
        sw      ra,28(sp)
        sw      s0,24(sp)
        addi    s0,sp,32
        sw      a0,-20(s0)
        sw      a1,-24(s0)
        lw      a4,-20(s0)
        lw      a5,-24(s0)
        add     a5,a4,a5
        mv      a0,a5
        lw      ra,28(sp)
        lw      s0,24(sp)
        addi    sp,sp,32
        jr      ra
```

07. SW Verification

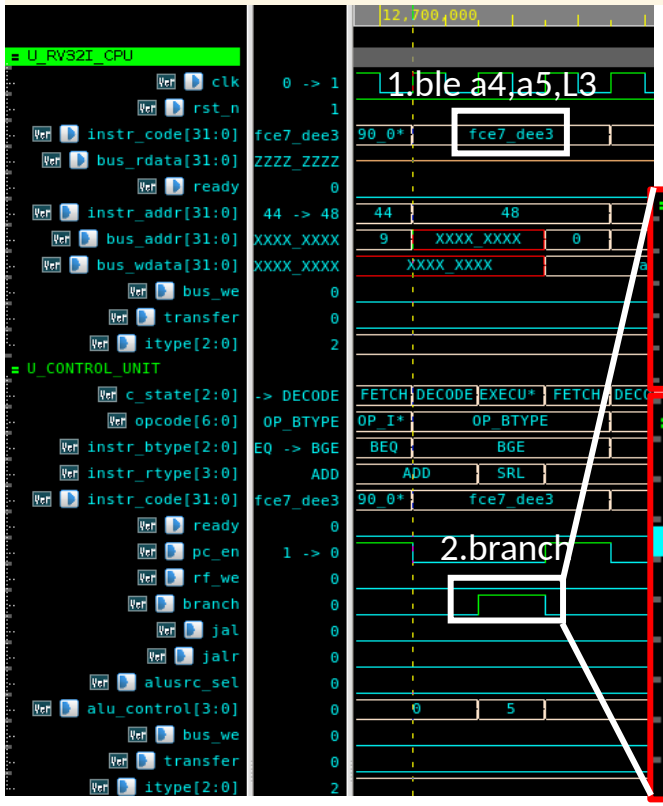
시뮬레이션 - Loop

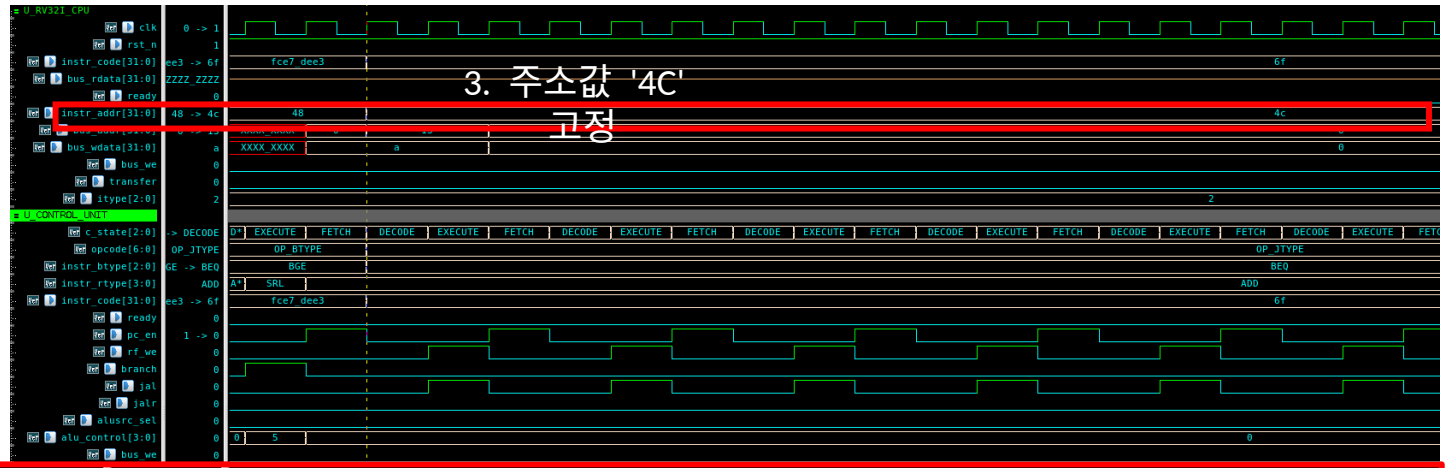
```
Assembly =   bge x15, x14, -36

  Binary =   1111 1100 1110 0111 1101 1110 1110 0011

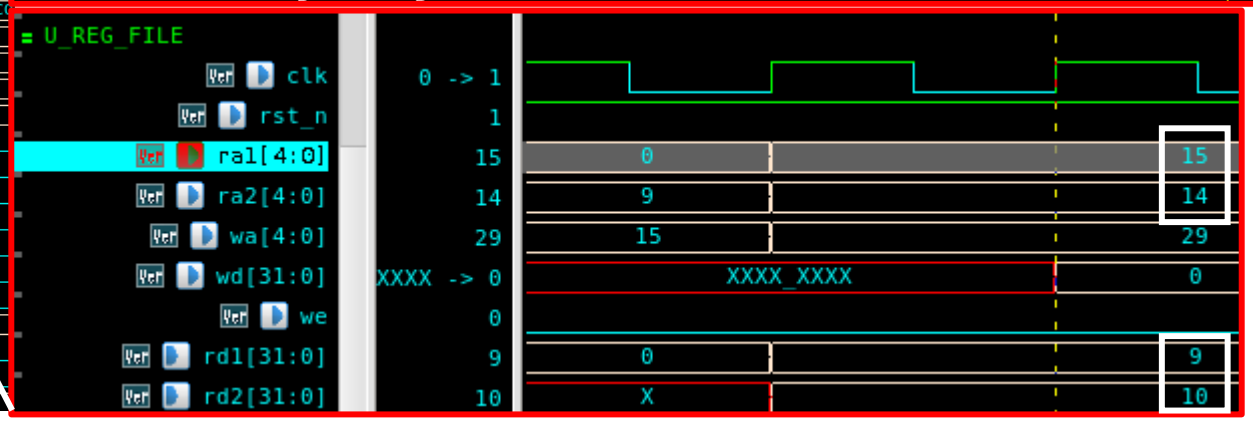
Hexadecimal = 0xfce7dee3

  Format =   B-type
```









False → b_taken(low)

08. Troubleshooting

■ 현상

```
[COV] ************************* rv32i_cpu Coverage Report *************************
[COV] Overall = 100.0% Reset = 100.0%
[COV] R commands = 100.0% I commands = 100.0% I shift commands = 100.0%
[COV] RS1 = 100.0% RS2 = 100.0% RD = 100.0%
[COV] R command x rs1 x rs2 = 100.0%
[COV] I command x imm = 100.0% I shift command x imm = 100.0%
[COV] R command x rd = 100.0%, I command x rd = 100.0%, I shift command x rd = 100.0%
[COV] ********************************************************************************
[SCB] PASS = 20896 FAIL = 332527
```

■ 결과

```
[COV] ************************* rv32i_cpu Coverage Report *************************
[COV] Overall = 100.0% Reset = 100.0%
[COV] R commands = 100.0% I commands = 100.0% I shift commands = 100.0%
[COV] RS1 = 100.0% RS2 = 100.0% RD = 100.0%
[COV] R command x rs1 x rs2 = 100.0%
[COV] I command x imm = 100.0% I shift command x imm = 100.0%
[COV] R command x rd = 100.0%, I command x rd = 100.0%, I shift command x rd = 100.0%
[COV] ********************************************************************************
[SCB] PASS = 541489 FAIL = 0
```

```
UVM_INFO ./tb/tb_rv32i_uvm.sv(889) @ 16239495000: uvm_test_top.env.SCB [SCB] PASS instr = 4147d193, rd = x3, expected_alu_result = 00000000, daddr = 00000000
UVM_INFO ./tb/tb_rv32i_uvm.sv(755) @ 16239505000: uvm_test_top.env.AGT.mon [MON] FETCH SRAI pc=00000244 instr=41d1da93 alu=0 rf_we=0 alusrc=0 daddr=00000000 dwdata=x
xxxxxxx
UVM_INFO ./tb/tb_rv32i_uvm.sv(755) @ 16239515000: uvm_test_top.env.AGT.mon [MON] DECODE SRAI pc=00000248 instr=41d1da93 alu=0 rf_we=0 alusrc=0 daddr=00000000 dwdata=
xxxxxxxx
UVM_INFO ./tb/tb_rv32i_uvm.sv(755) @ 16239525000: uvm_test_top.env.AGT.mon [MON] EXECUTE SRAI pc=00000248 instr=41d1da93 alu=d rf_we=1 alusrc=1 daddr=00000015 dwdata
=xxxxxxxx
UVM_ERROR ./tb/tb_rv32i_uvm.sv(897) @ 16239525000: uvm_test_top.env.SCB [SCB] FAIL instr=41d1da93, rd=x21, expected_alu_result = 00000000, daddr = 00000015, rf_we=1(
expected 1), alusrc=1(expected 1) alu_control = 1101(expected 1101)
```

08. Troubleshooting

■원인

결과 값을 상승 에지 1step 에서 인식

```
clocking mon_cb @(posedge clk);
    default input #1step;
    input rst_n, instr_code, instr_addr, alu_control;
    input daddr, dwdata, dwe, rf_we, alusrc_sel, itype;
endclocking
```

```
virtual task run_phase(uvm_phase phase);
    forever begin
        @(c_if.mon_cb);
        c_item = cpu_seq_item::type_id::create("c_item");
        c_item.rst_n = c_if.mon_cb.rst_n;
        c_item.sample_phase = phase_ref;
        c_item.instr_code = c_if.mon_cb.instr_code;
        c_item.command = cpu_decode(c_item.instr_code);
        c_item.instr_addr_before = c_if.mon_cb.instr_addr;
        c_item.alu_control = c_if.mon_cb.alu_control;
        c_item.dwe = c_if.mon_cb.dwe;
        c_item.rf_we = c_if.mon_cb.rf_we;
        c_item.alusrc_sel = c_if.mon_cb.alusrc_sel;
        c_item.itype = c_if.mon_cb.itype;
        c_item.instr_addr = c_if.mon_cb.instr_addr;
        c_item.daddr = c_if.mon_cb.daddr;
        c_item.dwdata = c_if.mon_cb.dwdata;
```

■해결방법

결과에 해당하는 값을 clocking 을

거치지 않고 상승 에지 1ns 뒤에 인식

```
virtual task run_phase(uvm_phase phase);
    forever begin
        @(c_if.mon_cb);
        c_item = cpu_seq_item::type_id::create("c_item"
        c_item.rst_n = c_if.mon_cb.rst_n;
        c_item.sample_phase = phase_ref;
        c_item.instr_code = c_if.mon_cb.instr_code;
        c_item.command = cpu_decode(c_item.instr_code);
        c_item.instr_addr_before = c_if.mon_cb.instr_ad
        c_item.alu_control = c_if.mon_cb.alu_control;
        c_item.dwe = c_if.mon_cb.dwe;
        c_item.rf_we = c_if.mon_cb.rf_we;
        c_item.alusrc_sel = c_if.mon_cb.alusrc_sel;
        c_item.itype = c_if.mon_cb.itype;
        #1;
        c_item.instr_addr = c_if.instr_addr;
        c_item.daddr = c_if.daddr;
        c_item.dwdata = c_if.dwdata;
```

08. 결론 및 고찰

1) 결론

(1) C 코드 기반 누적 sum 이 APB Bus 를 통해 BRAM 에 저장됨을 확인

(2) pass count = 541,489 예상값과 일치 , 계획한 coverage 검증률 100% 달성

2) 고찰

(1) 동작과 상관없는 신호들이 High 가 되는 것을 확인함으로써 , 향후 전력 소비 절감을 위한 최적화 필요

(2) APB Requester, Completer 추후 GPIO, UART 확장 까지 고려하여 설계

(3) coverage 구성의 불완전함 , R 타입의 경우 각 레지스터 (rs1, rs2) 에 대한 접근 보다
모든 값의 연산 여부를 체크할 필요가 있다 생각됨 추후 coverage 조건을 확장하여
검증의 신뢰성을 높이고자 함